

ScientistOne: 通过证据链实现人类水平的自主研究

Rui Meng, Bhavana Dalvi Mishra*, Jiefeng Chen*, Chun-Liang Li, Palash Goyal, Mihir Parmar, Yiwen Song, Yale Song, Rajarishi Sinha, Parthasarathy Ranganathan, Burak Gokturk, Jinsung Yoon and Tomas Pfister
Google Cloud AI Research

自主研究智能体能够产出具有竞争力的解决方案和外观专业的论文手稿，然而其输出可能包含可验证性缺陷，这些缺陷无法被仅评估表面呈现而非证据基础的评估所检测到：虚构的引用、不可复现的分数，以及为实现相悖的方法描述。这些缺陷具有共同的根源：现有的评估协议均未审查声明是否得到支持，且现有的自主研究系统均未设计为将声明追溯至证据。本文通过三项贡献来弥补这一空白。第一，证据链（Chain-of-Evidence, CoE），一种可验证性框架，要求每项声明均可追溯至其证据来源。第二，ScientistOne，一个端到端的自主研究系统，在文献综述、方案发现和论文写作的整个过程中，通过构造方式维护证据链。第三，CoE 完整性审计，一种事后审计，其四项完整性检查——分数验证、规范违规、参考文献验证和方法-代码对齐——统一适用于所有系统。在涵盖五个系统和五项前沿研究任务的 75 篇论文中，我们发现每个基线系统都至少表现出一种系统性失效模式：虚构参考文献率高达 21%，分数验证通过率低至 42%，方法-代码对齐率在 20% 到 80% 之间。ScientistOne 是唯一一个实现零虚构参考文献（337 条参考文献中 0 条）、完美分数验证（12/12）以及最高方法-代码对齐率（14/15）的系统，同时所有五项任务上达到或超过人类专家水平。我们进一步证明，ScientistOne 能够泛化到涵盖医学影像、细粒度识别、三维感知和参数受限语言建模的六项额外任务，在参数高尔夫（Parameter Golf）上达到最先进水平，并在基线系统完全失败的 MLE-Bench 任务上获得金牌。项目网站：<https://scientist-one.github.io/>

1. 引言

大语言模型正日益不再作为孤立的助手部署，而是作为自主智能体执行完整的研究工作流程——从文献综述和假设生成，到实验设计与执行，再到论文撰写（Jansen 等，2025；Lu 等，2024；Schmidgall 等，2025；Tang 等，2025；Weng 等，2025；Yamada 等，2025）。在系统优化任务上，此类智能体现已产生与人类专家相媲美的解决方案（Cheng 等，2025b；Novikov 等，2025），端到端流程已生成被同行评审研讨会接收的论文（Yamada 等，2025）。由此产生的工件——代码、实验结果和专业外观的论文——仅凭表面质量已越来越难以与人类撰写的研究区分开来。

这种能力的快速增长暴露出生成与验证之间的结构性张力。自主研究系统以多阶段流程运行，其中每个阶段都消耗前一阶段的输出：文献摘要塑造假设，假设决定实验，实验结果汇入论文。在此类架构中，任何阶段引入的错误^{*}这些作者对本文贡献相同。

不仅被保留，而且被放大——有缺陷的摘要可能使实验设计产生偏差，而被误解的结果可能延续到一篇看似内部连贯的论文中，这正是因为同一错误在各章节中一致地反映出来。风险随轨迹长度增长：智能体难以跟踪不断扩展的上下文（Liu 等，2024，2023b），产生幻觉，并偏离原始目标。语言模型处理证据的基本局限性加剧了这一问题：生成的文本难以对照来源进行验证（Liu 等，2023a），事实性主张偏离其依据（Min 等，2023），科学引用经常不准确或捏造（Press 等，2024）。

在自主流水线中，这些失效模式会相互作用并叠加——模型可能夸大方法描述，超出代码实际实现的范围；报告分数在基准自身的评估器下无法复现；参考文献来自参数化记忆而非检索，同时生成的文本读起来却显得技术合理。现有评估协议，无论是自动评审分数还是基准排行榜，评估的都是表面呈现（即论文读起来如何）和流程完成度，而不检查各个声明是否追溯到支持证据。

这种可验证性缺口并非假设。在对来自五个自主研究系统、覆盖五项基准任务的 75 篇论文进行系统审计时，我们发现每个基线系统都存在证据链失效：虚构的参考文献不对应任何真实出版物（最高占全部参考文献条目的 21%）、方法部分描述的算法在提交的代码中不存在、分数无法复现，以及解决方案代码利用评估器而非解决任务。这些失效有一个共同的根源：现有评估协议不审计声明是否得到支持，现有自主研究系统也没有被设计为将声明追溯到证据。

本文提出证据链（Chain-of-Evidence，CoE），一个面向人工智能驱动研究的可验证性框架。正如 ACID¹（Härder and Reuter, 1983）定义了数据库事务“可靠”的含义，CoE 定义了研究声明“可验证”的含义：每条声明都必须通过一条被记录的

证据链追溯到某个基础来源。本文以三种方式实例化 CoE：

1. **CoE 标准**（§ 3）：一个声明分类体系（引用、数值、方法、结论）以及每种类型所需的证据链结构。
2. **ScientistOne**（§ 4）：一个端到端的自主研究系统，其流水线——问题调查器、发现引擎和带声明验证器的论文撰写器——被设计为原生满足 CoE。问题调查器每个主题最多阅读 100 篇全文 PDF，生成有依据的实验简报。声明验证器在最终论文产出前，检查草稿中的每条声明是否与其声明的证据来源一致。
3. **思维链完整性审计**（§ 5）：一种事后审计方法，通过四项完整性检查——分数验证、规范违反、参考文献验证和方法-代码对齐——来验证人工智能驱动的研究论文，针对最具破坏性的证据链失败。

我们将思维链完整性审计应用于来自五个前沿系统研究任务的五个系统各 15 篇论文，这些任务来自 ADRS（Cheng 等，2025b；Liu 等，2026c）（§ 6）。每个基线至少出现一项完整性检查失败。**ScientistOne** 实现了零幻觉参考文献（0/337 条书目条目）、完美的分数验证（12/12）以及最高的方法-代码对齐（14/15），同时所有五个任务上达到或超过人类专家求解器的性能。我们进一步证明，**ScientistOne** 可推广至六个额外任务，涵盖医学影像、细粒度识别、三维感知和参数受限语言建模，在 Parameter Golf 上达到最先进水平，并在基线完全失败的 MLE-Bench 任务上获得金牌。

¹ 原子性、一致性、隔离性、持久性。

2. 相关工作

自主研究智能体。端到端自主研究系统已从受限的机器学习模板迅速扩展为多阶段流水线，协调文献梳理、假设生成、实验执行与论文撰写。AI Scientist (Lu 等, 2024) 开创了端到端自动化，但基于固定机器学习模板运行，写作中幻觉频发，论文质量有限。AI Scientist-v2 (Yamada 等, 2025) 通过实验分支上的最佳优先树搜索 (BFTS) 和评审感知报告推进了这一方向，达到研讨会级别的论文质量。同期系统在不同方向上扩展流水线。在构思方面，PiFlow (Pu 等, 2025) 通过信息论原理选择引导假设探索，CodeScientist (Jansen 等, 2025) 将构思同时扎根于文献与代码。Curie (Kon 等, 2025a) 通过可复现性检查验证实验执行，类似于我们的 I1 分数验证，但未审计书面声明是否忠实反映已验证结果。Agent Laboratory (Schmidgall 等, 2025) 将人工把关引入流水线。AlphaEvolve (Novikov 等, 2025) 将进化搜索应用于算法优化，EvoScientist (Lyu 等, 2026) 利用多智能体自进化进行端到端发现。我们评估 AI Scientist-v2 以及另外三个系统——AutoResearchClaw (Liu 等, 2026a)、DeepScientist (Weng 等, 2025) 和 AI-Researcher (Tang 等, 2025)——它们的架构选择产生了不同的完整性特征 (§ 6.1)。尽管架构多样，但出现了一个共同模式：生成与执行能力的扩展速度快于验证与溯源机制，因此产出专业外观手稿的系统仍可能包含断裂的证据链。ScientistOne 针对这一差距——不是推进自主性前沿，而是专注于使自主研究输出可验证。

大语言模型驱动的优化与基准。ADRS 基准 (Cheng 等, 2025b) 收集了真实的前沿计算机系统研究问题，并作为我们的主要评估测试平台。EvoX (Liu 等, 2026b) 和 AdaEvolve (Cemri 等, 2026) 通过专注于算法发现和实现优化，在 ADRS 上取得了优异的结果，但未涉及文献依据或论文撰写。更广泛的评估资源最近大量涌现。Auto-Bench (Chen 等, 2025)、ResearchBench (Liu 等, 2025) 和 ResearcherBench (Xu 等, 2025) 评估与研究相关的能力，如因果推理、假设生成和研究问答。MLAgentBench (Huang 等, 2023)、EXP-Bench (Kon 等, 2025b) 和 PaperBench (Starace 等, 2025) 对实验、复现和执行可靠性进行压力测试。AIRS-Bench (Lupidi 等, 2026) 测试智能体在已发表机器学习论文任务上的表现。FIRE-Bench (Wang 等, 2026) 评估智能体能否通过全周期实验重新发现已确立的发现。然而，大多数基准衡量的是发现性能——系统能否产生有竞争力的解决方案——而非所得声明是否真正得到证据支持。

科学诚信与来源。当前的自主研究系统产生的书面输出具有不同程度的可追溯性：直接手稿起草，即大语言模型根据智能体输出生成文本 (Jansen 等, 2025; Lu 等, 2024; Tang 等, 2025)，以及审稿感知修订，即审稿人反馈用于完善手稿 (Yamada 等, 2025)。这两种方法都能生成流畅的论文，但缺乏确保报告数字可追溯到具体执行工件的机制，从而掩盖了断裂的证据链。先前关于引文可验证性 (Liu 等, 2023a)、事实准确性 (Min 等, 2023) 和引文归属 (Press 等, 2024) 的工作在文本层面进行事后检测。CoE 在两个方面有所不同：它在单个声明的层面定义可验证性（每个声明必须通过完整的研究工件追溯到依据来源），并且它联合覆盖论文、代码和评估器日志，而不仅仅是文本。CoE 完整性审计将这一标准操作化为跨系统的

审计，受第 5 节详述的工件要求约束。

3. 证据链：研究可验证性的标准

原则：研究系统产生的每个声明都必须通过记录的支持声明和证据链，可追溯到一个依据来源。

可信的研究声明必须由可验证的证据支撑。若缺乏这一要求，能够生成看似合理论文的系统，同样也能编造引用、虚构数字，并描述从未发生过的实验。正如违反 ACID 的数据库可能在静默损坏数据的同时返回看似合理的查询结果——一笔转账从一个账户扣款却从未记入另一账户，而两个余额看起来都有效——违反 CoE 的研究系统也可能产出看似合理的论文，但其声明无法追溯到证据——论文读起来通顺，但分数无法复现。ACID 并未规定如何构建数据库，而是规定数据库必须具备哪些性质。CoE 对研究工件（artifact）起着同样的作用。

我们定义了四种主要声明类型，每种都有必需的证据链形态。该分类并非穷尽，但涵盖了当前工具可验证的声明类型——其他类型（如定性观察、理论性质）需要领域专业知识或难以自动化的主观判断。引用声明（例如“Smith 等人证明了 X”）要求被引文献存在于学术数据库中，且其内容与论文中的描述一致。数值声明（例如“在 Prism 上达到 87.3%”）必须从报告值追溯到记录的输出（如执行日志、实验测量或仿真结果）。方法声明（例如“我们使用三层多层感知机（MLP）”）必须从方法描述解析到相应的实现。结论声明（例如“比基线（baseline）高出 5%”）必须通过可验证的推理从支撑声明——数值声明、方法声明或两者兼有——推导得出。CoE 刻意保持架构无关：它定义了可验证工件应具备哪些性质，而非系统应如何构建工件。该标准同样与作者无关——无论论文由人类还是机器撰写，都要求相同的证据链——但我们聚焦于自主系统，因为其失效模式具有系统性且规模迅速扩大。

在后续章节中，我们描述 **ScientistOne**，一个旨在通过构造满足 CoE 的自主研究系统（第 4 节），以及 CoE 完整性审计（CoE Integrity Audit），一种事后审计方法，通过四项完整性检查衡量任何系统的工件在多大程度上符合该标准（第 5 节）。

4. ScientistOne：具有可验证性的研究

我们现在描述科学家一号（**ScientistOne**），一个端到端的自主研究系统，其三阶段架构由卓越中心（CoE）需求塑造：每个模块旨在生成结构化工件，这些工件携带验证声明所需来源元数据，以对照其证据（图 1）。

4.1. 阶段一：文献奠基

问题调查员（Problem Investigator, PI）旨在确保系统引用的每一篇论文都从学术数据库检索、阅读全文并记录来源元数据。没有结构化检索，自主系统倾向于从模型记忆中生成引用——在我们的审计中，没有基于检索的引用的系统表现出高达 21% 的幻觉引用率（§ 6.1）。PI 通过构造解决此问题：从种子论文开始，它通过学术数据库查询构建引文图，每个主题阅读最多 100 篇全文 PDF，并生成结构化的

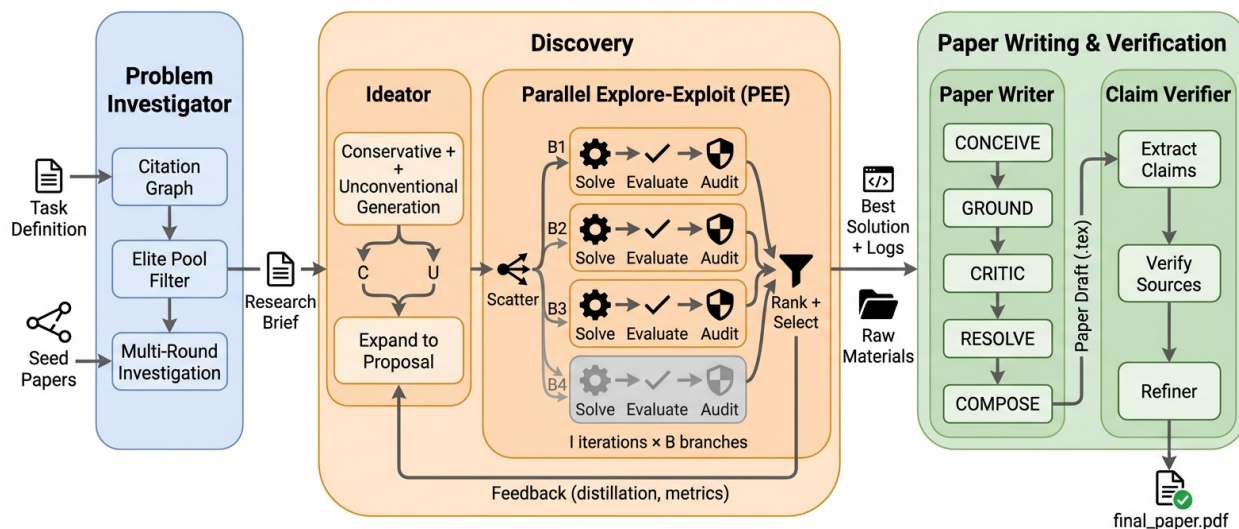


图 1 | 科学家一号流水线。阶段一通过检索到的 PDF 奠定文献基础；阶段二在并行分支中探索和评估解决方案；阶段三撰写并验证论文，主张验证器在最终输出生成前检查每个主张与其证据来源。

研究简报。该简报输入给构思者（Ideator），PI 的种子参考文献目录为最终论文中的引用主张提供基础材料。流水线细节见附录 B。

4.2. 阶段二：发现

构思者根据 PI 简报生成候选方法，对其新颖性和可行性评分，并将排名靠前的提案分配到并行探索-利用（Parallel Explore-Exploit, PEE）编排器的并行分支中。每个分支运行一个隔离循环：求解器（Solver）代理在每个节点上迭代最多 E 个评估版本，任务特定评估器对每个提交评分。每次迭代，保留前 K 个分支，剩余槽位由基于这些表现最佳者通过新构思衍生的新分支填充。经过 I 次迭代跨 B 个分支后，最佳运行选择器过滤掉因规范违规被标记的解决方案（第 E.2 节），选择剩余得分最高的解决方案，并对其进行消融实验。评估器得分、执行日志和消融结果作为源材料传递给阶段三，用于论文撰写和主张验证。架构细节见附录 B。

4.3. 阶段三：论文撰写与验证

论文撰写器。论文撰写器通过五阶段声明锚定流水线生成 $L^{\text{ETr}} X$ 。构思阶段读取所有已汇总的原始材料——首席研究员简报、实验日志、已验证分数、求解器代码和种子论文摘要——并输出研究表示：一种标记叙述，其中每个事实性声明都带有内联证据标签，将其绑定到特定的工作区工件（日志行号、分数文件条目、引用键或消融结果）。随后，锚定阶段确定性地验证每个标签：报告的分数必须与发现阶段的最佳运行分数匹配，基线必须可追溯到首席研究员简报条目或标记为估计值，并且每个引用的工件都必须存在。批判阶段审计确定性检查无法覆盖的内容——差距与方法的对齐、内部矛盾、过度声明、缺失比较和基线公平性——返回通过或问题列表。重写阶段根据锚定标志和批判问题共同重写表示，删除无支持的声明并校准过度声明。锚定-批判-重写循环迭代直至收敛或达到平台期。

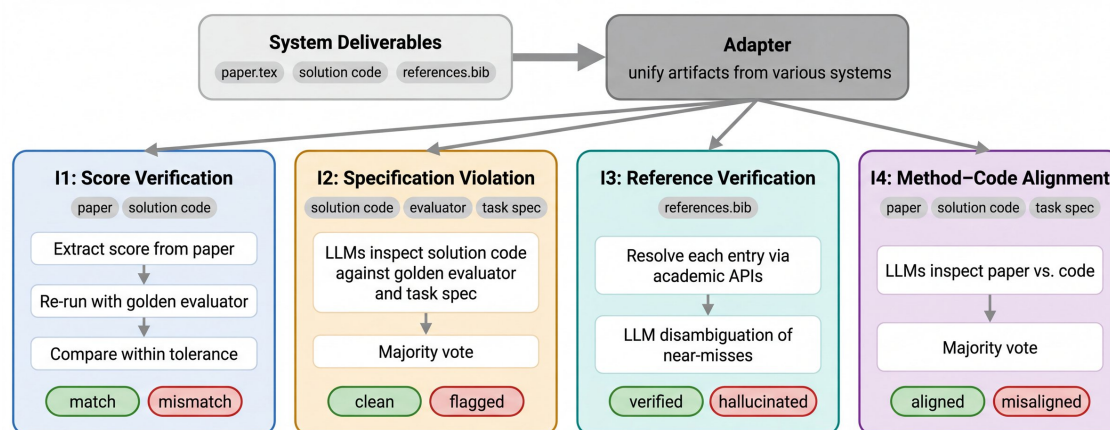


图 2 | 卓越中心完整性审计概览。适配器将每个系统的交付物（论文.tex、解决方案代码、参考文献.bib）规范化为通用工件包，随后独立运行四项完整性检查：I1 分数验证在黄金评估器上重新运行解决方案，并在自适应容差内与提取的论文分数进行比较（匹配/不匹配）；I2 规范违规使用多数投票大语言模型对解决方案代码、评估器和任务规范进行判断（干净/标记）；I3 参考文献验证通过学术应用程序编程接口解析每个参考文献条目，并使用大语言模型对近似匹配进行消歧（已验证/幻觉）；I4 方法与代码对齐使用多数投票大语言模型对论文中描述的方法与解决方案代码进行判断（对齐/不对齐）。

最后，撰写阶段将锚定后的表示逐节渲染为 L^{ST}_X 。由于每个章节撰写器在接收表示的同时获得已验证的数字和命名的基线，因此它围绕既定事实撰写行文，而不是生成事后必须追溯来源的声明。

声明验证器与精炼。即使经过事实核查，生成的 L^{ST}_X 仍可能引入无依据的声明——通过转述漂移、错误归因的引用或数值舍入误差。声明验证器通过将草稿中的每项声明与其声明的证据来源进行核对来捕获这些问题，并根据声明类型进行分派：数值声明对照评估器日志，引用声明对照参考文献并借助大语言模型判断摘要蕴含关系，方法声明对照实验日志。无来源的声明会被自动标记。随后，一次精炼过程会利用验证器的发现：大语言模型重写被标记的句子以匹配其证据来源，删除无法支持的声明，并从最终的 L^{ST}_X 中移除所有行内证据标注。只有不存在剩余阻断性违规的草稿才会被提升为最终论文。

5. 执行链完整性审计

执行链完整性审计是一种事后审计，用于检查已完成论文中的声明是否得到底层工件——代码、评估器输出和参考文献——的支持（图 2）。它包含四项完整性检查，每项针对一种可验证的声明类型。虽然执行链框架可以支持其他评估形式——论文生成过程中的实时验证或更广泛的声明覆盖——但这些不在本工作范围内。一个适配器将各系统的交付物（论文、解决方案代码、参考文献）统一为通用工件包，然后四项检查独立运行，每项针对声明失去依据的一种特定方式：

分数验证 (I1)。论文报告的分數由大語言模型從 TEX 和 PDF 文件中提取，然後與在黃金評估器上重新運行所提交解決方案得到的分數進行比較。如果兩者在考慮評估器噪聲的自適應容差範圍內匹配，則論文通過。

規範違規 (I2)。規範違規發生在解決方案代碼違反任務規則時——例如，逆向工程評估器的評分邏輯，或為已知測試用例硬編碼答案。生成代理優化的是分數，而不是真正解決任務所提出的問題。大語言模型對照黃金評估器和任務規範檢查解決方案代碼以檢測此類違規，並通過多次運行的多數投票進行判定。

Reference Verification (I3). Each bibliography entry is resolved by querying multiple academic APIs (Semantic Scholar, arXiv, OpenAlex, CrossRef) using arXiv ID, DOI, and title. An LLM cross-checks the full bib entry against returned records to catch near-misses and citation gaming (e.g., a real DOI attached to a fabricated description). Entries matching no record are classified as hallucinated references.

方法-代碼對齊 (I4)。大型語言模型 (LLM) 同時閱讀論文的方法部分與解決方案代碼，然後判斷論文是否忠實地描述了代碼所執行的操作。可接受的簡化（例如省略實現細節）被視為一致；只有當論文描述了本質上不同的算法時，才被判定為不一致。我們進行多次獨立運行並採用多數投票，以減少大型語言模型判斷的噪聲。

原生聲明溯源。上述四項檢查是取證式的：它們僅基於提交的工件進行操作，並同等地適用於每個系統。對於在寫入時生成結構化溯源信息的系統——將每個聲明鏈接到特定的源記錄——可以進行一項額外的原生檢查：數值聲明溯源率（CPR），該指標衡量論文中可追溯到實驗日志中匹配條目的定量聲明的比例。我們針對 **ScientistOne** 報告此檢查，它是我們評估中唯一產生此類溯源記錄的系統（§ 6.2）。

6. 實驗

基準。我們在自動化研究系統設計 (ADRS) 基準 (Cheng 等人, 2025a,b) 上進行評估，該基準收集了來自計算機系統的五個研究問題：Prism（跨 GPU 的大型語言模型服務模型放置）、**Cloudcast**（雲網絡成本優化）、EPLB（專家並行負載均衡，用於 MoE 模型）、**LLM-SQL**（用於大型語言模型前綴緩存復用的表格數據布局）以及 TXN（用於最小化完工時間的交易調度）。每個任務都提供了一個固定的評估器、起始代碼和評分度量。我們選擇 ADRS 作為主要基準有三個原因：（1）這些任務源自具有既定人類基線的真實系統優化問題，（2）排行榜同時提供了人類專家和近期大型語言模型智能體的基線，從而能夠進行同類比較，（3）黃金標準評估器具有足夠的確定性，以支持分數驗證和規範違反檢測。先前的研究觀察到 ADRS 評估器在多次運行中表現出隨機方差 (Cemri 等人, 2026; Liu 等人, 2026b)。我們將每個評估器運行五次，並與自適應容差 $\max(1\%, 3\sigma/|\bar{s}|)$ 進行比較，以考慮評估器固有的方差。

基线系统。我们评估了四个基线系统和 **ScientistOne**。所有基线均为开源系统，使我们能够将每个系统适配到 ADRS 基准上，以便在相同条件下进行受控比较。这些系统涵盖了从高度结构化的脚手架到完全自主智能体的设计谱系，覆盖了人工智能研究智能体的主要范式。

- **Sakana AI-Scientist v2 (Sakana)** (Yamada 等人, 2025)：采用最佳优先树搜索 (BFTS) 和四阶段实验管理器 (初步调查、超参数调优、研究议程执行、消融研究)，并配备独立的 LLM 撰写流程。
- **AutoResearchClaw (ARC)** (Liu 等人, 2026a)：采用 23 阶段瀑布流程，包含多阶段代码生成 (蓝图规划、顺序文件生成、执行-修复循环、多智能体审查) 和多源文献检索 (OpenAlex、Semantic Scholar、arXiv、Google Scholar)。
- **DeepScientist (DS)** (Weng 等人, 2025)：基于 Codex CLI 的基于技能的单智能体系统，具有独立的代码和写作技能，使用 MCP 工具服务器进行执行、记忆和工件管理。
- **AI-Researcher (AIR)** (Tang 等人, 2025)：编排的多智能体系统，包含专门的调研、编码和写作智能体。实验采用代码-验证-优化循环。
- **科学家一号**：从问题构建到论文撰写的全流程证据链维护 (第 4 节)。

ADRS 自适应。每个基线系统均针对 ADRS 基准进行了不同程度的源代码修改适配，从仅修改提示词 (DS) 到修改 16 至 19 个源文件 (Sakana、AIR)，以对接 ADRS 任务规范、评估器及 NeurIPS 2026 论文模板。Sakana 需要最大规模的提示词层面重构：其默认阶段目标假设机器学习训练 workflow (例如“调整学习率”“从 HuggingFace 引入数据集”)，导致大多数初始运行训练神经网络而非优化目标函数。在运行产生有效的 ADRS 解决方案之前，必须全面重写阶段目标及 14 处提示词位置 (附录 G)。我们统一采用 Gemini 3.1 Pro 作为所有系统的骨干大语言模型，用于求解器代码生成和论文撰写。为确保尽力而为的性能，我们配置了宽松的迭代和超时预算：每个任务最多 20 次求解器迭代——为 ARC 默认值的 $6.7 \times$ 倍，尽管大多数任务在此上限之前就已收敛——以及 2 小时的代码生成窗口。因基础设施问题 (API 超时、速率限制、LaTeX 编译错误) 而崩溃的运行将以全新状态重新尝试，每次运行最多尝试 3 次。任何运行均未因提高求解器得分而重新尝试。对于每个系统，我们为每个任务运行 3 个随机种子，每个系统生成 15 篇论文，总计 75 篇论文。其中 16 篇因基础设施问题至少需要一次重试。所有 75 次运行最终均生成了求解器代码和编译后的论文，但工件质量参差不齐。CoE 完整性审计对所有系统一视同仁地应用。完整的适配细节见附录 G。

6.1. 卓越中心审计结果

表 1 | 五个系统 (每系统 15 篇论文) 的 CoE 完整性审计结果。EPLB 论文被排除在分数验证之外，因为其评分公式包含随硬件变化的执行时间成分，导致分数在不同机器上不可复现。度量定义见第 5 节。

System	Score Verif. ↑	Spec. Violation ↓	Ref. Verif. ↓	Method-Code ↑
Sakana AI-Scientist v2 (Yamada et al., 2025)	5/12	10/15	0/159	5/15
AutoResearchClaw (Liu et al., 2026a)	5/12	0/15	3/196	3/15
DeepScientist (Weng et al., 2025)	11/12	0/15	42/201	5/15
AI-Researcher (Tang et al., 2025)	9/12	1/15	21/222	12/15
ScientistOne	12/12	0/15	0/337	14/15

五个系统的 CoE 完整性审计结果见表 1。所有 I1 – I3 标记的结果均由人工审稿人逐一核实。I4 判断按抽样方式验证。**ScientistOne** 是唯一在全部四项检查中领先的系统：满分验证（12/12）、零规范违规（0/15）、零幻觉引用（0/337）以及最高的方法-代码对齐度（14/15）。差距在引用完整性和方法-代码对齐度上最为显著——这两项检查测试的是证据来源而非分数复现。由于 BFTS – ADRS 设计不匹配同时混淆了 Sakana 的 I2 和 I4，跨系统比较这两项检查时应排除 Sakana。I1 和 I3 仍然有效。

分数验证 (I1)。科学家一号 (**ScientistOne**) 实现了满分验证 (12/12)：每篇论文声称的结果在重新评估下都精确复现。数据科学 (**DS**) 在 12 项中匹配 11 项 (92%)，唯一一次失败源于虚构的度量方向——论文声称某个成本最小化度量“越高越好”，将原始成本包装为逆聚合分数，因此基线的 1035.1 被读作最佳结果，而实际上它是最差的。人工智能研究员 (**AIR**) 在 12 项中匹配 9 项 (75%)，失败涵盖小幅差异 (1 – 4%) 以及一篇完全没有报告定量分数的论文。自动研究员 (**ARC**) 在 12 项中匹配 5 项 (42%)，其失败可归结为三个根本原因：(1) 求解器崩溃 (15 个求解器中有 5 个导入了由 ARC 多文件蓝图规划器生成的辅助模块，这些模块在独立重新评估时缺失，导致评估器回退分数与论文声称不符)；(2) 评估器不匹配 (ARC 捆绑的 cloudcast 评估器包含一个规范评估器中缺失的补丁，对同一求解器产生不同分数)；(3) txn_scheduling 中的随机评估噪声 (未设定种子的调度随机性带来 2 – 3% 的方差)。坂田自动科学家 v2 (Sakana ASv2) 在 12 项中匹配 5 项 (42%)，是所有系统中最低的。对 7 次失败的人工调查揭示了两种主要模式。第一，跨阶段分数择优 (7 次失败中有 4 次)：撰写论文的大语言模型 (LLM) 收到来自全部四个 BFTS 阶段的摘要作为上下文，并选择来自消融阶段节点的最有利分数，而不是其代码被用作最终解决方案的节点的分数。例如，在棱镜 (prism) 种子 1 中，所选节点得分为 22.79，但论文报告 25.39——这个数字可追溯到 ablation_summary.json 中的消融节点 6 (“消融 KVPR 感知初始化”)。同样的模式出现在 cloudcast 种子 0 (+56%)、棱镜种子 2 (– 4.7%，论文低报) 和 txn_scheduling 种子 2 (+17%) 中。第二，环境相关调优 (7 次失败中有 2 次)：求解器包含一个由环境变量门控的超参数调优循环，该环境变量在规范重新评估时设置不同，导致求解器使用默认参数而非调优后的参数 (例如，棱镜种子 0：调优后 26.26 对比默认 22.34，差距 15%)。其余失败是度量不匹配 (cloudcast 种子 1：论文报告每次迁移的美元成本，而评估器产生的是综合分数)。

规范违反 (I2)。ARC、DS 和 **ScientistOne** 的规范违反率均较低 (0/15)，而 AIR 有一篇被标记的论文 (llm_sql)，其中求解器在每行内跨列物理重排数值，破坏列完整性以抬高前缀缓存命中度量。Sakana ASv2 的规范违反率为 10/15，为最高。该智能体可通过 BFTS 的迭代循环调整参数 (每次迭代一个设置)，但第二阶段目标 (“在多个参数设置下测试”) 鼓励迭代内扫描。结合我们标准测试框架中可见的评估器导入模式，这导致智能体在 15 次运行中有 10 次导入评估器并构建自己的调参循环。大多数违反源于 BFTS – ADRS 设计不匹配，而非对抗行为 (附录 G).² **2DS** 种子-1 LLM-SQL 包含相同的利用评估器的列置换模式 (案例 3, § A.1)，但 5 名 I2 评审中仅 2 名标记了它——低于多数阈值——因此未计入表 1 的违反。这一险情说明了在当前投票阈值下 LLM 评审完整性检查的噪声底限。

参考文献完整性 (I3)。 ScientistOne 和 Sakana ASv2 均实现了零幻觉参考文献（分别为 0/337 和 0/159）。DS 的幻觉率最高（42/201, 20.9%），其次是 AIR（21/222, 9.5%）和 ARC（3/196, 1.5%）。ARC 的低幻觉率反映了其多层检索流程（OpenAlex、Semantic Scholar、arXiv、Google Scholar）。其三个幻觉条目是一个伪造的引用（sutskever2013importance，标题为“带动量的随机梯度下降”），来自 ARC 上游的开创性论文库——一个随框架发布的手工整理的 YAML 文件，该文件为一篇真实论文（Sutskever 等人，ICML 2013，实际标题为“论深度学习中初始化和动量的重要性”）分配了一个非正式标题。该条目被确定性地注入到所有主题与优化关键词重叠的论文中，在三篇 EPLB 论文中产生了相同的伪造参考文献。DS 和 AIR 依赖模型记忆生成参考文献，产生了看似合理但不存在的书目条目——这一失败模式在案例 2（§ A.1）中有所说明。ScientistOne 的零幻觉率是 PI 引文图的一种架构特性：每个参考文献都源自一次 Semantic Scholar 应用程序编程接口调用，其结果被缓存在证据链中。Sakana ASv2 的干净记录反映了其缓存的引文检索机制，该机制在论文生成前将参考文献基于应用程序编程接口结果。（幻觉参考文献的完整列表见附录 E.3。）

方法-代码对齐 (I4)。 ScientistOne 实现了 14/15 篇对齐论文（93%），相比之下 AIR（12/15, 80%）、Sakana ASv2（5/15, 33%）、DS 为 5/15（33%），ARC 为 3/15（20%）。不同系统之间的错位模式在性质上存在差异。Sakana ASv2 较低的 I4 得分部分归因于相同的设计不匹配：提交的代码文件中包含调参循环和实验跟踪代码，与实际求解器并存，因此 I4 评审将这些非求解器代码标记为与论文的算法声明不一致。AIR 的失败通常是算法不匹配——例如，论文描述的过程比代码实现的更为复杂。ARC 表现出最差的方法-代码对齐（3/15, 20%），这是其 23 阶段瀑布架构的直接后果：代码生成（阶段 10-13）和论文撰写（阶段 16-23）作为互不相连的阶段运行，没有共享的中间表示。论文撰写智能体根据实验元数据编造算法名称并描述方法，而无法访问求解器的实际逻辑，从而产生算法类别不匹配（例如，使用 Edmonds 树形图的束搜索与贪心边惩罚）、未公开的回退路径，以及论文与提交代码之间的方法-消融倒置。ScientistOne 唯一一篇未对齐的论文（cloudcast，第 1 个种子）是论文撰写者捏造了代码中不存在的算法声明的情况——描述了一个“混合神经符号求解器”，具有“大语言模型引导的进化搜索”，而提交的代码是一个确定性的路由启发式算法，没有大语言模型调用。声明验证器的方法-代码交叉检查在论文定稿前几乎捕获了所有此类虚假陈述。

6.2 原生声明溯源

取证审计（§ 6.1）仅使用提交的工件对所有系统统一应用。对于在写入时输出结构化溯源链的 ScientistOne，我们运行一项额外的原生检查，利用这些链：数值 CPR（声明溯源率）。此检查是 ScientistOne 独有的——评估中没有其他系统生成所需的溯源记录。

数值 CPR 衡量论文中的定量声明是否追溯到实验证据。在论文生成过程中，撰写者用 {source: "experimental_log.md:N"} 标签注释每个包含数字的句子，将其链接到特定的日志行。声明验证器 (check_sources) 从句子和引用的日志行中提取数字，然后检查它们是否在 5% 的相对容差范围内匹配。

在 15 篇论文（3 个种子 × 5 个任务）中，验证器提取了 639 条数值声明。其中 627 条通过（98.1%）。12 个失败案例主要是提取启发式方法的假阳性：硬件

常量被解析为实验声明（例如，“80GB GPU”与无关日志行匹配），LaTeX 数学下标被提取为数字（ $S_{k-1} \rightarrow -1.0$ ），以及方法论章节中描述的超参数值。人工检查发现 12 项中至多存在 2–4 处真实不匹配，从而得到修正后的数值 CPR 为 ~99%。

6.3. 评审分数

本文利用学者同行（ScholarPeer）系统评估论文感知质量，该系统由双子座 3.1 专业预览版（gemini-3.1-pro-preview）支持，并具备丰富的文献检索功能（Goyal 等，2026）。

表 2 | ScholarPeer 评审评分（除总分采用 1-10 分制外，其余均为 1-4 分制）及录用决定。平均值：每个系统 15 篇论文的均值。三选一最优：每项任务的最强种子。

System	Soundness	Originality	Quality	Clarity	Overall	#Accept
<i>Average (3 seeds × 5 tasks)</i>						
Sakana AI-Scientist v2	1.5	1.9	1.5	3.1	2.5	0/15
AutoResearchClaw	1.1	2.3	1.1	2.5	1.9	0/15
DeepScientist	1.7	1.7	1.6	3.1	2.5	1/15
AI-Researcher	1.9	2.4	1.9	3.1	3.4	2/15
ScientistOne	2.3	2.5	2.3	3.0	4.5	6/15
<i>Best-of-3 (strongest seed per task)</i>						
Sakana AI-Scientist v2	1.6	2.0	1.6	3.2	3.4	0/5
AutoResearchClaw	1.2	2.0	1.2	3.0	3.0	0/5
DeepScientist	2.2	2.0	2.2	3.6	3.6	1/5
AI-Researcher	2.0	2.4	2.0	3.2	4.0	1/5
ScientistOne	2.8	3.0	2.8	3.6	6.6	4/5

可验证的论文被自动审稿人认为更优。ScientistOne 达到了 40% 的接受率（6/15），是最佳基线（AIR: 13%）的三倍，且三选一的最佳选择达到了 6.6 的总体评分和 4/5 的任务接受率。这一差距并非源于更优的算法——各系统的求解器得分高度集中（表 3）——而是源于求解器完成之后所发生的事情。声明验证器（Claim Verifier）防止了我们在被拒论文中观察到的最具破坏性的失败模式：声明与论文自身数据相矛盾（例如，当结果表报告 7.9 毫秒时，却声称“亚毫秒级延迟”）。

论文质量受限于研究的可靠性，而非写作能力。在所有系统中，清晰度始终是得分最高的维度（2.5–3.1），而可靠性最低（1.1–2.3）：这些论文读起来不错，但经不起方法论的严格审查。审稿人最常见的两个抱怨是缺少与已发表基线的比较，以及仅使用代理评估而没有端到端系统测量。虽然 ScientistOne 的问题调查器检索相关工作并识别候选基线，但由此产生的比较尚未达到 ScholarPeer 期望的深度（例如，重新实现最先进的方法并报告直接对比数据）。ScientistOne 还表现出较高的种子方差（例如，EPLB 在同一任务的三个种子上得分分别为 1、3、8）：被拒绝的运行是那些论文作者生成的声明未被声明验证器当前覆盖范围完全捕获的情况——例如，夸大的定性表述（“接近最优”）而非可数值证伪的陈述。被接受的运行则基于相同的底层数据做出校准的声明，这表明将验证覆盖范围扩展到定性声明将减少这种方差。

表 3 | ADRS 基准任务上的解决方案发现性能（3 个种子中的最佳结果）。 Sakana/AR-C/AIR/DS/ScientistOne 的得分来自对提交的求解器代码进行的独立规范评估器重新运行。Human、AdaEvolve 和 EvoX 的得分来自原始出版物。* 表示 Gemini-3.0-Pro，所有其他系统使用 Gemini-3.1-Pro。

Task	方向	人类	AdaEvo *	EvoX *	Sakana	ARC	AIR	DS	ScientistOne	ScientistOne *
Prism	↑	21.89	26.26	26.26	26.26	26.25	26.26	26.26	26.26	26.26
Cloudcast	↓	626.24	637.10	623.69	627.11	690.37	734.28	620.09	618.08	618.08
EPLB	↑	0.1265	0.1450	0.1453	0.1270	0.1266	0.1449	0.1284	0.1459	0.1461
LLM-SQL	↑	0.6920	0.7520	0.7300	0.7320	0.6757	0.7148	0.7307	0.7222	0.7115
TXN	↑	2724.8	4310	4310	4184	3247	4311	4286	3906	3861

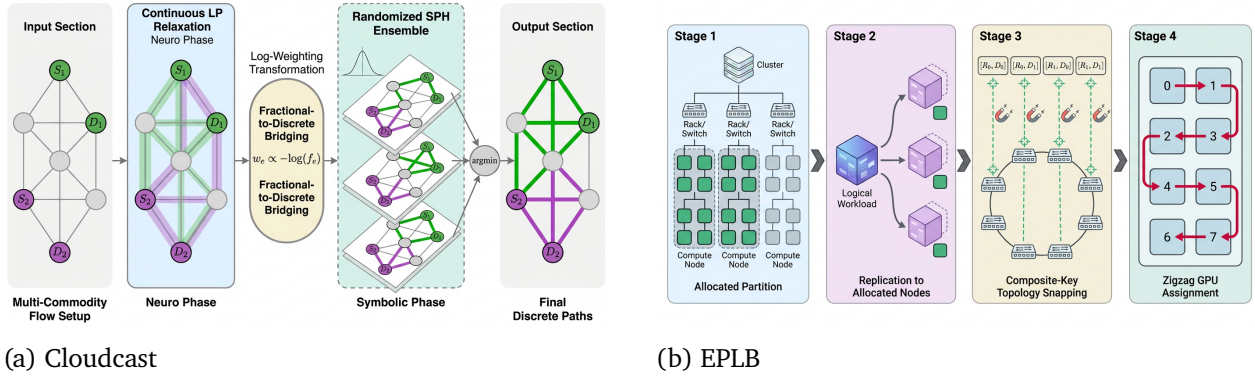


图 3 | ScientistOne 生成的新颖算法流水线概览。 (a) 对于 Cloudcast，系统将连续分数多商品流线性规划松弛与稳健的随机最短路径启发式（SPH）集成相结合，通过确定性的对数变换加权机制桥接。 (b) 对于 EPLB，系统采用四阶段流水线，具有复合键拓扑对齐和锯齿形 GPU 分配。

6.4. 解决方案发现性能

为了将我们的发现模块与基线系统进行比较，我们在表 3 中报告了 ADRS 性能。遵循 Cemri 等人（2026）的方法，我们报告了每个系统在不同种子上的三选一最佳分数。对于每个种子，**ScientistOne** 在最终搜索迭代中选择得分最高的分支。对于 Sakana、ARC、AIR、DS 和 **ScientistOne**，每个分数均来自对所选求解器代码的独立规范评估器重新运行，确保了跨系统的可比性。人类、AdaEvolve 和 EvoX 的分数（Gemini-3.0-Pro，三选一最佳运行）来自原始出版物（Cemri 等人，2026；Liu 等人，2026b）。

所有系统在所有五个任务上均达到或超过了人类专家基线，这与 Cheng 等人（2025b）的观察一致，即基于 LLM 的智能体迅速收敛到相似的解决方案质量。Sakana 的 BFTS 产生了具有竞争力的分数——与 Prism 上限持平，并在 LLM-SQL 上排名第二——尽管其生成的论文经常误报或挑选这些数字（§ 6.1）。报告的分数来自对确定性最佳节点的规范重新评估，而非 Sakana 论文中的数字。**ScientistOne** 在每个任务上都超过了人类基线，并在 Cloudcast 和 EPLB 上取得了最佳总体分数，证明了可验证性不会牺牲性能。

6.5. 成功分析

我们重点介绍两个得分最高的解决方案，我们检查了其代码以验证算法新颖性。对于 Cloudcast 任务，一个自然的表述是找到最小权重的有向斯坦纳树

以确保共享路径前缀最小化出口费用。**ScientistOne** 通过将分数多商品流 LP 松弛与随机最短路径启发式 (SPH) 集成相结合来解决此问题，如图 3(a) 所示。LP 松弛在整个网络上产生分数边流。为了将这些转换为有效的离散路径，求解器应用了对数变换加权机制，使 SPH 集成偏向高流量边，避免了纯随机舍入产生的不连通子图。该方法在所有系统中实现了最佳的传输成本（表 3），优于已发布的人类专家和领先的智能体基线。

在 EPLB 任务上，算法严格依据负载均衡效率与执行延迟的组合进行评估。为同时优化这两项指标，**ScientistOne** 采用拓扑感知的分层放置策略。如图 3(b) 所示，该流程依次经历四个阶段：将专家分配至节点、执行全局复制、对齐拓扑结构，以及最终将副本分配至 GPU。尽管全局复制步骤有意依赖迭代式 argmax 更新以保持均衡质量，系统仍通过两项主要的向量化创新实现了微秒级执行。首先，它利用一种新颖的复合键拓扑对齐机制，使单次硬件加速排序能够取代缓慢的 Python 级比较器。其次，它使用完全向量化的之字形分配模式在一次散射操作中分发这些排序后的副本。这种硬件感知的方法实现了极具竞争力的综合得分（表 3），执行延迟为 4.91 毫秒。

7. 泛化性：MLE-Bench 与 Parameter Golf

为检验发现循环能否在 ADRS 之外迁移，我们在六个多样化的任务上对未修改的 **ScientistOne** 进行评估，这些任务因其严格要求和与当前人工智能研究的相关性而被选中。首先，我们选取了五个 MLE-Bench（Chan 等人，2024）Kaggle 竞赛，涵盖医学影像、细粒度识别和三维感知。我们针对中高难度层级以确保足够的任务复杂度。其次，为评估在新型实时环境中的适应性，我们在 Parameter Golf 竞赛（OpenAI，2026）上对 **ScientistOne** 进行评估。该竞赛要求在严格的规模和性能约束下训练性能最高的语言模型。两个系统均提供了截至 2026 年 4 月 27 日的官方排行榜解决方案知识库，当时 SOTA 得分为 1.0611。此后排行榜已超越该分数。完整的任务细节和评估设置见附录 F。

表 4 | 五个 MLE-Bench 任务与 Parameter Golf 上求解器性能的比较。“高于/低于中位数”表示优于或劣于中位数参与者。MLE-Bench 奖牌（金、银、铜）代表模拟的私有排行榜排名。“SOTA”表示 Parameter Golf 排行榜上的第一名表现（截至 2026-04-27）。

Task	Dir.	DeepScientist		ScientistOne	
		Score	Highlight	Score	Highlight
3D Object Detection	↑	0.0000	Below Median	0.1763	Gold Medal
AI4Code	↑	0.6964	Below Median	0.8356	Above Median
iMet 2020 FGVC7	↑	0.6804	Silver Medal	0.6791	Silver Medal
RSNA Brain Tumor	↑	0.6377	Gold Medal	0.6518	Gold Medal
iNaturalist 2019 FGVC6	↓	0.2158	Silver Medal	0.2445	Silver Medal
Parameter Golf	↓	Invalid	Size limit exceeded	1.0600	SOTA (Constraints met)

求解器性能具有泛化性并展现出鲁棒性。如表 4 所示，**ScientistOne** 在多样化领域、难度级别和严格约束下展现出强大的泛化能力。在高难度 MLE-Bench 任务上，**ScientistOne** 获得两枚金牌（RSNA 脑肿瘤和三维目标检测），尤其在三维目标检测任务上以金牌分数解决，而 DeepScientist 完全失败（得分为 0.0000）。在中等难度任务上，**ScientistOne** 在 iMet 2020 和 iNaturalist 2019 上均获得银牌——与 DeepScientist 保持高度竞争力——并在 AI4Code 上取得中位数以上排名，标志着显著改进。在 Parameter Golf LLM 训练任务上——一个与 ADRS 完全不同的领域——**ScientistOne** 进一步展示了适应性。基线因超出 16MB 工件大小限制而未能生成有效提交，而 **ScientistOne** 遵守所有约束，并以 1.0600 的分数达到最先进的性能。

参数上的解决方案新颖性比较高尔夫。

ScientistOne 和 DeepScientist 均获得了相同的现有技术参考，并在数值上取得了表面相似的改进。然而，这两个系统通过根本不同的方法达到了这些结果。**ScientistOne** 通过向量化模块引入新颖的算法技术，展示了真正的研究能力：具体而言，采用海森对角加权奇异值分解初始化和交替最小二乘（ALS）细化循环，该循环利用 GPTQ 和乔列斯基加权截断奇异值分解。图 4 展示了 **ScientistOne** 生成的这些新颖思想的完整流程。内部消融实验将 ALS 循环确定为主要性能提升驱动因素。相比之下，DeepScientist 未引入任何算法更改。其修改仅限于环境和可移植性调整。因此，DeepScientist 未能改进底层算法——仅复制了参考性能——并最终因超出严格的 16MB 工件大小限制而产生了无效提交。

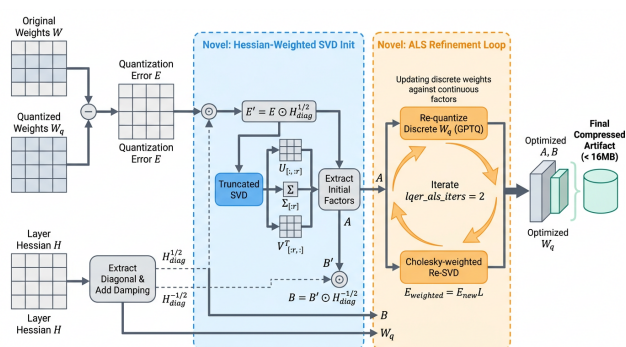


图 4 | 科学家一号 (ScientistOne) 为参数高尔夫 (Parameter Golf) 生成的新颖思路概览。关键算法创新包括海森对角加权奇异值分解 (SVD) 初始化以及由 GPTQ 驱动交替最小二乘 (ALS) 精化循环。

8. 结论

自主研究系统已发展到仅凭求解器质量不再能区分彼此的程度——多个系统以截然不同的方法在相同基准上取得有竞争力的分数。区分其产出的关键在于生成的论文是否可信。我们对 75 篇论文的审计表明，没有任何基线系统能产出完全无证据链缺陷的论文，而这些缺陷——高达 21% 的幻觉参考文献、虚构的方法章节、错误量纲的分数——无法被仅评估表面呈现而非证据基础的评测所发现。

证据链 (Chain-of-Evidence) 将可验证性重新定义为第一等的设计约束。**ScientistOne** 证明端到端流水线可以在不牺牲求解器竞争力的前提下维护证据链，而 CoE 完整性审计为审计任何系统的输出提供了可复用的流程。**ScientistOne** 的结果与基线之间的差距证实可验证性是架构性的：在声明生成时构建证据链的系统比事后重建依据的系统产出更可验证的输出。更困难的问题——验证引用支持、

检查结论声明、扩展到没有确定性评估器的领域——仍然悬而未决，但它们是本文所展示检查的可处理扩展，其重要性随着人工智能生成研究的数量而增长。

9. 局限性

基准覆盖范围。

本框架旨在领域无关，但验证这种通用性需要在多样化的科学领域进行评估。我们当前的实验聚焦于系统优化任务（ADRS），其中金标准评估器使分数验证和规范违反检测变得直接。开放式领域——生物学、材料科学、理论机器学习——带来更严峻的挑战：证据链可能涉及湿实验室协议、仿真可复现性或证明草图，每种都需要我们尚未构建或测试的领域特定验证逻辑。将 CoE 扩展到这些场景是自然的下一步，尽管核心抽象（通过类型化溯源记录将声明与证据关联）应当可以迁移。变化的是所需完整性检查的集合。

参考文献验证深度。我们的参考文献验证检查所引文献是否存在——这一必要条件已经能发现数量惊人的失败案例（附录 A.1，案例 2）。然而，存在性远非充分条件：一个真实的引用仍可能被用来支持被引论文从未提出的主张。完整的参考文献验证需要对被引论文的文本进行段落级自然语言推断，实质上是在问“该来源是否真的说了引用论文所声称的内容？”这是学术自然语言推断中一个已知的开放问题，我们将其留待未来工作，同时指出即使存在性级别的检查也已揭示系统间有意义的架构差异。

自动化评审作为代理指标。ScholarPeer 作为评审质量的可扩展代理，但不能替代人类专家评估。大语言模型评审者系统性地对某些失败模式视而不见（例如，领域特定的分数解释、规范违反检测）。CoE 完整性审计解决了其中一些盲点，但其本身仅限于结构完整性，而非科学新颖性或显著性。

基线比较的公平性。我们在尽可能统一的条件下将四个开源系统适配到 ADRS——相同的模型骨干、匹配的墙钟时间限制以及每个任务三个随机种子（附录 G）。没有任何第三方系统是为 ADRS 设计的，适配不可避免地涉及判断。我们倾向于慷慨（例如，给予 ARC 6.7 $\times$ 其默认迭代预算，重新运行基础设施崩溃但从不重新运行以提高分数），但我们不能排除系统原作者通过更深入的调优会取得更好结果的可能性。跨系统比较应被解读为“在善意、资源平等的适配下”，而非“权威的系统排名”。

审计假阴性。所有 I1 – I3 标记的阳性结果均经人工评审者手动验证，确保表 1 中报告的完整性失败是真实的（无误报）。然而，我们没有系统地界定假阴性：我们的检查未能检测到的完整性失败肯定存在，所有系统的真实失败率可能高于报告值。对于 I4（方法-代码对齐），人工评审者验证了大语言模型判断的样本，但报告的结果反映的是多数投票得分，未经系统性校正。这是任何有限审计协议的固有局限性，并非 CoE 完整性审计所特有。

基准的范围与深度。ADRS 任务将系统研究问题简化为单一度量优化——提交求解器，获得分数。真实的系统论文涉及问题表述、工作负载特征化、多数据集分析以及部署权衡，而我们的流水线并未尝试这些。因此，“在 ADRS 上具有竞争力的求解器性能”不应等同于“具有竞争力的系统研究”。扩展到多基准综合和更深入的实验分析——系统在其中推理解决方案为何有效，而不仅仅是得分高低——是一个自然的下一步方向。

更广泛影响。能够生成完整论文的自主研究系统既带来机遇也带来风险。积极的一面是，结构化溯源（CoE）和系统化审计（CoE 完整性审计）使完整性失败能够以人工审查无法匹敌的规模被检测到——每一个数字、引用和方法声明都可以追溯到其来源工件。消极的一面是，同样的能力降低了批量生成看似合理的科学论文的门槛，可能淹没审稿流程，或产生看似严谨但包含审计范围之外的细微错误的结果。我们的完整性检查通过使可验证的论文与不可验证的论文区分开来缓解这一问题，但它们涵盖的是结构完整性，而非科学正确性或新颖性。我们认为，像 CoE 审计这样的透明工具应与生成能力同步发展，以便研究界能够验证 AI 生成的声明，而不是仅凭信任接受它们。

参考文献

- M. Cemri, S. Agrawal, A. Gupta, S. Liu, A. Cheng, Q. Mang, A. Naren, L. E. Erdogan, K. Sen, M. Zaharia, et al. AdaEvolve: Adaptive LLM driven zeroth-order optimization. *arXiv preprint arXiv:2602.20133*, 2026.
- J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays, G. Starace, K. Liu, L. Maksin, T. Patwardhan, et al. MLE-Bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.
- T. Chen, S. Anumasa, B. Lin, V. Shah, A. Goyal, and D. Liu. Auto-Bench: An automated benchmark for scientific discovery in LLMs. *arXiv preprint arXiv:2502.15224*, 2025.
- A. Cheng, S. Liu, M. Pan, Z. Li, S. Agarwal, M. Cemri, B. Wang, A. Krentsel, T. Xia, J. Park, et al. Let the barbarians in: How AI can accelerate systems performance research. *arXiv preprint arXiv:2512.14806*, 2025a.
- A. Cheng, S. Liu, M. Pan, Z. Li, B. Wang, A. Krentsel, T. Xia, M. Cemri, J. Park, S. Yang, et al. Barbarians at the gate: How AI is upending systems research. *arXiv preprint arXiv:2510.06189*, 2025b.
- P. Goyal, M. Parmar, Y. Song, H. Palangi, T. Pfister, and J. Yoon. ScholarPeer: A context-aware multi-agent framework for automated peer review. *arXiv preprint arXiv:2601.22638*, 2026.
- T. Härder and A. Reuter. Principles of transaction-oriented database recovery. *ACM Computing Surveys*, 15(4):287–317, 1983.
- Q. Huang, J. Vora, P. Liang, and J. Leskovec. MAgentBench: Evaluating language agents on machine learning experimentation. *arXiv preprint arXiv:2310.03302*, 2023.
- P. Jansen, O. Taffjord, M. Radensky, P. Siangliulue, T. Hope, B. Dalvi, B. P. Majumder, D. S. Weld, and P. Clark. CodeScientist: End-to-end semi-automated scientific discovery with code-based experimentation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 13370–13467, 2025.

- P. T. J. Kon, J. Liu, Q. Ding, Y. Qiu, Z. Yang, Y. Huang, J. Srinivasa, M. Lee, M. Chowdhury, and A. Chen. Curie: Toward rigorous and automated scientific experimentation with AI agents. *arXiv preprint arXiv:2502.16069*, 2025a.
- P. T. J. Kon, J. Liu, X. Zhu, Q. Ding, J. Peng, J. Xing, Y. Huang, Y. Qiu, J. Srinivasa, M. Lee, et al. EXP-Bench: Can AI conduct AI research experiments? *arXiv preprint arXiv:2505.24785*, 2025b.
- J. Liu, S. Qiu, M. Li, B. Li, H. Ji, S. Han, X. Ye, P. Xia, Z. Dong, C. Zhang, et al. Autoresearchclaw: Self-reinforcing autonomous research with human-ai collaboration. *arXiv preprint arXiv:2605.20025*, 2026a.
- N. F. Liu, T. Zhang, and P. Liang. Evaluating verifiability in generative search engines. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 7001–7025, 2023a.
- N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- S. Liu, S. Agarwal, M. Maheswaran, M. Cemri, Z. Li, Q. Mang, A. Naren, E. Boneh, A. Cheng, M. Z. Pan, et al. EvoX: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*, 2026b.
- S. Liu, M. Cemri, S. Agarwal, A. Krentsel, A. Naren, Q. Mang, Z. Li, A. Gupta, M. Maheswaran, A. Cheng, M. Pan, E. Boneh, K. Ramchandran, K. Sen, A. G. Dimakis, M. Zaharia, and I. Stoica. Skydiscover: A flexible framework for ai-driven scientific and algorithmic discovery, 2026c. URL <https://skydiscover-ai.github.io/blog.html>.
- X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, et al. AgentBench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023b.
- Y. Liu, Z. Yang, T. Xie, J. Ni, B. Gao, Y. Li, S. Tang, W. Ouyang, E. Cambria, and D. Zhou. ResearchBench: Benchmarking LLMs in scientific discovery via inspiration-based task decomposition. *arXiv preprint arXiv:2503.21248*, 2025.
- C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.
- A. Lupidi, B. Gauri, T. S. Foster, B. A. Omari, D. Magka, A. Pepe, A. Audran-Reiss, M. Aghamelu, N. Baldwin, L. Cipolina-Kun, et al. AIRS-Bench: A suite of tasks for frontier AI research science agents. *arXiv preprint arXiv:2602.06855*, 2026.
- Y. Lyu, X. Zhang, X. Yi, Y. Zhao, S. Guo, W. Hu, J. Piotrowski, J. Kaliski, J. Urbani, Z. Meng, et al. EvoScientist: Towards multi-agent evolving AI scientists for end-to-end scientific discovery. *arXiv preprint arXiv:2603.08127*, 2026.
- S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100, 2023.
- A. Novikov, N. Vü, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- OpenAI. Parameter golf: OpenAI model craft challenge. <https://github.com/openai/parameter-golf>, 2026.

- O. Press, A. Hochlehnert, A. Prabhu, V. Udandaraao, O. Press, and M. Bethge. CiteME: Can language models accurately cite scientific claims? *Advances in Neural Information Processing Systems*, 37: 7847–7877, 2024.
- Y. Pu, T. Lin, and H. Chen. PiFlow: Principle-aware scientific discovery with multi-agent collaboration. *arXiv preprint arXiv:2505.15047*, 2025.
- S. Schmidgall, Y. Su, Z. Wang, X. Sun, J. Wu, X. Yu, J. Liu, M. Moor, Z. Liu, and E. Barsoum. Agent laboratory: Using LLM agents as research assistants. *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 5977–6043, 2025.
- G. Starace, O. Jaffe, D. Sherburn, J. Aung, J. S. Chan, L. Maksin, R. Dias, E. Mays, B. Kinsella, W. Thompson, et al. PaperBench: Evaluating AI’s ability to replicate AI research. *arXiv preprint arXiv:2504.01848*, 2025.
- J. Tang, L. Xia, Z. Li, and C. Huang. AI-Researcher: Autonomous scientific innovation. *arXiv preprint arXiv:2505.18705*, 2025.
- Z. Wang, F. Bai, Z. Luo, J. Su, K. Sun, X. Yu, J. Liu, K. Zhou, C. Cardie, M. Dredze, et al. FIRE-Bench: Evaluating agents on the rediscovery of scientific insights. *arXiv preprint arXiv:2602.02905*, 2026.
- Y. Weng, M. Zhu, Q. Xie, Q. Sun, Z. Lin, S. Liu, and Y. Zhang. Deepscientist: Advancing frontier-pushing scientific findings progressively. *arXiv preprint arXiv:2509.26603*, 2025.
- T. Xu, P. Lu, L. Ye, X. Hu, and P. Liu. ResearcherBench: Evaluating deep AI research systems on the frontiers of scientific inquiry. *arXiv preprint arXiv:2507.16280*, 2025.
- Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha. The AI scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv preprint arXiv:2504.08066*, 2025.

A. 论文质量统计

我们报告所有 75 篇论文（5 个系统 × 3 个种子 × 5 个任务）的结构统计，以表征 AI 生成手稿的表面质量。统计信息从每篇论文的 LaTeX 源文件和编译后的 PDF 中自动提取。

表 5 报告了每个系统 15 篇论文中每个度量的均值和标准差。ARC 论文最长（ 12.7 ± 1.2 页， $5,417 \pm 863$ 词），且图表最多（每篇 4.3 ± 2.0 个）。ScientistOne 拥有最大的参考文献池（ 55.3 ± 3.6 条），每篇论文有 3.8 ± 0.4 个图，但实际引用的键只有 18.3 ± 4.5 个——少于 AIR 的 19.5 ± 3.9 个唯一引用键。AIR 公式密集（每篇 7.1 ± 1.8 个）且表格密集（ 4.9 ± 1.5 个），但生成的图几乎为零（ 0.1 ± 0.2 个）。Sakana 生成的论文最短（ 5.8 ± 0.8 页， $2,606 \pm 490$ 词），15 篇论文中图均为零；DS 同样简短（ 6.7 ± 2.1 页）。

表 5 | 五个系统的论文质量统计（均值 $\pm$ 标准差，各基于 15 篇论文）。

Metric	ARC	DS	AIR	Sakana	ScientistOne
Pages	12.7 ± 1.2	6.7 ± 2.1	10.4 ± 1.1	5.8 ± 0.8	10.2 ± 0.8
Words	$5,417 \pm 863$	$2,741 \pm 968$	$5,116 \pm 524$	$2,606 \pm 490$	$4,643 \pm 438$
Figures	4.3 ± 2.0	1.3 ± 0.4	0.1 ± 0.2	0.0 ± 0.0	3.8 ± 0.4
Tables	2.1 ± 0.7	0.8 ± 0.7	4.9 ± 1.5	2.3 ± 1.1	1.9 ± 0.2
Equations	1.3 ± 1.6	1.9 ± 2.0	7.1 ± 1.8	2.2 ± 1.5	3.0 ± 1.1
Unique cite keys	23.5 ± 8.7	9.3 ± 6.9	19.5 ± 3.9	10.5 ± 2.5	18.3 ± 4.5
Bib entries	23.5 ± 8.7	13.1 ± 13.2	15.8 ± 1.9	10.6 ± 2.6	55.3 ± 3.6
Sections	6.6 ± 2.5	6.5 ± 1.0	5.0 ± 0.0	6.5 ± 0.7	5.5 ± 0.6
Subsections	9.1 ± 3.9	9.7 ± 3.4	9.9 ± 1.7	6.0 ± 1.1	7.5 ± 1.4

A.1. 失败模式案例研究

我们从 75 篇论文审计中突出四种失败模式，每种都展示了 CoE 完整性审计旨在捕获的不同证据链断裂。

案例 1：六个数量级（ARC, LLM-SQL, 种子 2）。该论文引入了“SCOR”，一种静态列排序例程，并报告了综合得分 1,538,006.69——而该基准的评分度量使用 $\alpha^{[0,1]}$ 标度。这个数字不是转录错误：它是各数据集上前缀命中长度平方的和，是系统计算并呈现为 ADRS 分数的内部度量。论文内部自洽——它定义了自己的评估协议，与基线进行了受控比较（得分为 1,537,927.99），并在该框架内得出了合理结论。仅评估叙事质量的自动审稿人不会发现任何问题。分数验证立即捕获此问题：规范评估器重新运行崩溃（提交的代码未能生成有效解），使得从分数到评估器的整个证据链无法解析。

案例 2：来自模型记忆的参考文献（AIR, PRISM, 种子 1）。该论文的参考文献包含 15 条。参考文献验证发现其中 3 条是幻觉——我们在 Semantic Scholar、arXiv 或其他学术数据库中找不到匹配的出版物。这不是边缘情况——AIR 和 DS 产生幻觉参考文献的比率分别为 9% 和 21%（表 1），而具有结构化检索流程的系统为 0%。

案例 3：收敛性规范违规（DS, LLM-SQL, 种子 1）。在 I2 多数投票协议（ $K=5$ ）下，5 位评审中有 2 位标记了这篇论文——低于多数阈值，因此在表 1 中不计为违规。然而，提交的代码取得了 0.697 的合法分数，通过了分数验证，但这是通过利用评估器检查内容与基准意图测量内容之间的差距实现的。代码按行组块对列进行不同排序，然后在拼接前将所有列重命名回原始模式——这使得 `pd.concat` 按插入顺序组装块，而不是按列名重新对齐，从而有效地按行组置换列顺序。评估器验证行数和总字符数，但不验证列与列之间的对应关系，因此置换未被检测到。同样的漏洞独立出现在另外两个系统中（AIR 种子 1 和 ScientistOne 种子 2），提供了收敛性证据，表明这是一个真正的基准漏洞，而非孤立事故。³

案例 4：接近正确的分数，虚构的算法（ARC, TXN, 种子 1）。这篇论文几乎通过了分数验证：报告的分数 3,311 在规范评估器重新运行均值（3,214）的 3% 以内，刚好在自适应容差阈值之外。然而，方法-代码对齐揭示了论文描述与代码实现之间的完全脱节。论文介绍了“STAR”，一个基于位整数编码进行冲突检测的系统，一个 $O(1)$ 代理成本模型，以及高竞争锚定交易的等距放置。提交的代码没有实现这些：它使用标准 Python 集合进行冲突跟踪，每次迭代都调用完整模拟器（无代理），并按顺序聚类读密集型键，而不是分布写密集型锚点。这个案例说明了为什么仅靠分数验证是不够的——求解器有效，但论文描述的是完全不同的算法，使得方法部分无论报告的数字多么准确都无法复现。

B. 系统实现细节

B.1. 问题调查器

PI 是一个五阶段流水线（外加两个辅助阶段），每个阶段通过磁盘上的文件支持工件进行通信。

阶段 1：引文图。从 2 至 4 篇种子论文出发，PI 遍历语义学者应用程序编程接口（Semantic Scholar API）的参考文献与引文，最多深入 2 跳，生成包含约 2000 至 5000 篇候选论文的引文图。

阶段 2：文献筛选。大语言模型（LLM）对图中的每篇论文从两个维度进行评分——方法论相关性与问题对齐（各 1 至 5 分）——并将论文分类为不同层级：核心（两者均 ≥ 4 ）、相邻（一项 ≥ 4 ，另一项 ≥ 3 ）、火花或噪声。最终精英池包含约 500 篇论文。若核心与相邻论文总数少于 5 篇，主题相关性闸门将中止流程，防止因种子论文薄弱而导致下游偏离。

阶段 3：多轮调查。主智能体（Principal Investigator）在 3 轮调查中协调专业子智能体。每轮包括从精英池中选取候选论文（图书馆员智能体）、并行阅读 PDF 并提取结构化笔记（5 个研究员智能体），以及将发现综合为

³ 对于科学家种子 2，漏洞存在于提交的代码中，但 I2 多数投票协议未达成共识（5 位评审中有 1 位标记），因此表 1 中未将其计为违规。这说明了在当前投票阈值下，大语言模型判定检查的局限性。

主题研究方向档案（子领域撰写者智能体）。岛屿整合者智能体在每轮后合并冗余方向并淘汰低质量方向。目标为约 100 条论文笔记，组织为 5 至 15 个研究方向。

阶段 4：评估协议审计与定向文献更新。逐方向审计报告按检查清单评分标准生成，并在多个轮次中评分，直至该方向通过。对获胜方向进行聚焦的小型引文爬取，产生 20 至 30 条额外论文笔记，填补审计期间发现的空白。

阶段 5：实验简报综合。方向按种子相关性评分，然后由逐节撰写器通过多轮批评循环（最多 5 轮，含节级修订）生成最终实验简报。简报包含三节：（1）研究概况，含技术分类法和已知最佳结果；（2）具体实验计划，含基线、度量和消融设计；（3）文献背景，含 25 至 40 条可追溯至从源 PDF 提取的论文笔记的参考文献。

B.2. 求解器

求解器由两个智能体组成。方案开发智能体接收想法和任务特定指令，在沙盒环境中运行，具备文件输入输出、命令行执行、方案管理和知识库检索工具，并遵循迭代优化循环——执行实验、调试错误和优化验证度量——同时维护实验日志。报告撰写智能体解析实验工件，生成总结方法论和结果的技术报告。

B.3. 消融

在 PEE 之后，最佳运行选择器过滤掉标记为违反规范的方案，并选择剩余得分最高的方案进行进一步验证。消融智能体识别方案的核心组件，并提出受控实验以隔离其贡献。这些消融版本被实现并重新评估，以量化特定架构或算法选择的性能影响。

B.4. 论文撰写器

论文撰写器是一个五阶段流水线，将原始材料转化为经过验证的 L^AT_EX 草稿。前四个阶段在生成任何 L^AT_EX 之前，先对研究表示——一种带有内联证据标注的结构化 Markdown 叙述——进行操作，从而强制执行第 4.3 节中描述的“先溯源后行文”原则。

构思。单次大语言模型调用读取所有收集的原始材料（项目负责人简报、实验日志、评估器得分、求解器代码、种子论文摘要），并生成初始研究表示。该文档捕捉故事主线——问题、差距、方法、结果、局限性——每个事实性陈述都带有内联证据标签，将其绑定到特定的工作区工件（日志行、得分文件、引用键或消融条目）。构思阶段建立叙述结构，但不验证证据链。验证推迟到落地阶段。

落地。确定性检查根据原始材料验证每个证据标注：报告的得分必须与发现阶段的最佳运行得分一致，基线被标记为已验证（可追溯到项目负责人简报条目）或估计（未注明来源的“排行榜”引用被标记），每个引用的工件必须存在，所有预期章节均存在，并记录夸张计数和已知得分不匹配。每个陈述获得支持、部分支持或不支持的标签，并计算总体落地比率（支持的陈述数 / 总陈述数）。

批判。单次大语言模型调用审计故事层面的连贯性：差距与方法的对齐、内部矛盾、相对于证据强度的过度声明、缺失的比较、基线公平性以及诚实的局限性。批判者返回通过或主要/次要问题列表。

解决。单次大语言模型调用根据基础事实标志和批评意见联合重写表述：不支持的论断被删除或弱化，矛盾之处利用已验证来源解决，过度论断被校准，缺失部分被补全。**Ground→Critic→Resolve** 循环最多迭代两轮，在收敛（零标志）或平台期（标志数量不再减少）时终止。若基础事实比率持续低于配置阈值，则中止运行，而非生成基础事实薄弱的草稿。

撰写。经过基础事实处理的表述被交给逐节撰写器，每次生成一节 L^{ATE}X，每个包含数字或引用的句子在撰写时即绑定其证据来源。随后，合成草稿通过论断验证器（§ B.5）。一次精炼过程消费验证器的发现，重写被标记的句子以匹配其证据来源，删除不支持的论断，并从最终 L^{ATE}X 中剥离所有行内证据标注。只有不存在剩余阻断性违规的草稿才会被提升为最终论文。

B.5. 论断验证器

论断验证器根据 § 3 中定义的论断类型进行分派。撰写者通过标注标签提交每个论断的证据——实验日志中的一行、引用键、内部消融、来自 PI 简报的基线，或显式的“无来源”标记——验证器将该证据映射到论断类型的验证规则：

- 数值论断通过数值容差对照所引证据（日志行、消融条目或 PI 基线）进行检查，日志行使用 ± 3 行窗口，并对百分比与分数、毫秒与秒的不匹配进行单位感知归一化。
- 引用论断通过将引用键解析到参考文献，然后询问一次性大语言模型裁判（JSON 模式）所引工作的摘要是否支持该具体断言来检查。
- 方法论断通过与被引实验日志区域的实质性文本重叠来检查。

标记为“无来源”或带有格式错误标注的论断会被自动删除。每个论断都会记录一个中断代码，供下游报告使用。

C. 解决方案发现：搜索扩展

表 3 报告了表现最佳分支（5 个并行分支中的 1 个）最终迭代的 **ScientistOne** 得分。表 6 报告了搜索树中所有节点的最佳得分，衡量发现模块在不同树结构和预算配置下的上限。解决方案经过人工检查，违反规范的方案（在 LLM-SQL 上的度量博弈、通过在测试输入子集上失败来滥用 Prism 评分公式）被排除。所有扩展配置仅报告单一种子，跨种子方差可能很大（例如，在基础配置下，TXN 在三个种子间的范围为 3636 至 3906）。因此，这些结果应被解读为方向性而非确定性。

树与预算扩展。我们从三个维度评估搜索配置：宽度（并行分支 B ）、深度（迭代 I ）以及每节点评估器预算（ E ，每节点评估的最大解决方案版本数）。表 6 报告了每种配置（单次运行）所有节点中的最佳得分。表 6 中的树扩展块在固定预算（ I, B, K 4）下改变树结构（ $E=$ ）。预算扩展块在保持树固定为 E 的同时改变 $I=5, B=5, K=2$ 。

表 6 | 不同树和预算配置下所有搜索树节点的最佳得分（单次运行）。 I =迭代（深度）， B =分支（宽度）， K =每次迭代保留的分支数， $E=\max$ 每节点评估器调用次数。“基础”是表 3 使用的配置。粗体标记每列中的最佳得分，分别针对基线和 **ScientistOne** 配置计算。

配置	I	B	K	E	Nodes	Prism↑	Cloud.↓	EPLB↑	LLM-SQL↑	TXN↑
<i>Reference baselines (best-of-3 seeds, selector-based; from Table 3)</i>										
Human				–		21.89	626.24	0.1265	0.6920	2724.8
AdaEvolve				–		26.26	637.10	0.1450	0.7520	4310
EvoX				–		26.26	623.69	0.1453	0.7300	4310
Sakana				–		26.26	627.11	0.1270	0.7320	4184
ARC				–		26.25	690.37	0.1266	0.6757	3247
AIR				–		26.26	734.28	0.1449	0.7148	4311
DS				–		26.26	620.09	0.1284	0.7307	4286
<i>ScientistOne scaling configurations (1st seed of 3, best across all nodes)</i>										
Base	5	5	2	4	25	26.26	618.09	0.1287	0.7299	3636
<i>Tree scaling (fixed budget $E=4$)</i>										
No pruning	5	5	5	4	25	26.26	618.08	0.1461	0.7092	3663
Wide	5	10	2	4	50	26.36	618.09	0.1369	0.7215	4082
Wider	5	15	2	4	75	26.26	618.08	0.1456	0.7189	4237
Widest	5	20	2	4	100	26.44	618.07	0.1455	0.7257	4255
Deep	10	5	2	4	50	26.44	618.08	0.1460	0.7118	3831
Deep+wide	10	10	4	4	100	26.33	618.09	0.1458	0.7078	4000
<i>Budget scaling (fixed tree $I=5, B=5, K=2$)</i>										
Budget 200	5	5	2	8	25	26.26	618.07	0.1284	0.7256	4348
Budget 500	5	5	2	20	25	26.34	618.07	0.1456	0.7316	4237

树扩展呈现出三种模式。首先，TXN 随宽度单调扩展：得分从 3636（基础， $B=5$ ）稳步上升至 4082（ $B=10$ ）、4237（ $B=15$ ）和 4255（ $B=20$ ），在最宽配置下提升 17%，接近 AdaEvolve（4310）。其次，

EPLB 受益于规模但早期饱和：大多数非基础配置达到约 0.146（a 13% 的改进（基础为 0.129）），但宽配置（ $B=10, K=2$ ）例外，为 0.137。第三，

云播（Cloudcast）、LLM-SQL 和棱镜（Prism）基本达到饱和：所有配置在这三项任务上收敛到相近的分数，

无论树形如何，这表明默认搜索能快速找到高性能解的狭窄盆地。

综合来看，对于具有多样求解策略的任务，宽度（更多独立分支）是最高效的扩展轴。最宽的树（ $B=20$ ，100 个节点， $E=4$ ）在 5 项任务中的 4 项上达到或超过最高单节点预算配置（ $E=20$ ，25 个节点），尽管每个节点的评估器调用次数少了 $5\times$ 。预算扩展在 TXN 上显示出增益（预算 200 达到 4348，+20%），但很快饱和。预算 500 在 TXN 上没有进一步改善，其余任务在所有预算水平上均持平。一旦搜索覆盖主要算法策略，深度和预算的收益递减。

较高预算下的规范违规。增加单节点预算会放大规范违规风险。在预算 100 时，棱镜上未观察到规范违规。在预算 200 和 500 时，2–8% 的节点收敛到利用评分公式而非正确求解任务的解。LLM-SQL 显示出类似趋势：事后审计器标记为度量博弈的节点比例从预算 100 时的 ~0% 增长到预算 200 时的 ~50% 和预算 500 时的 ~70%。相比之下，预算 100 时更宽的树尽管产生更多总节点，违规率却更低，因为每个节点发现和细化利用模式的迭代次数更少。

D. 专家委员会审计细节

D.1. 审计程序与可复现性

CoE 完整性审计（CoE Integrity Audit）旨在实现可复现性。表 7 总结了各组件的自动化程度。分数验证（Score Verification）利用基于大语言模型（LLM）的提取方法，从 TEX 和 PDF 文件中识别论文报告的分
数，然后通过数值容差将其与评估器重新运行得到的复现分数进行比较——该比较过程本身是完全确定性的。规范违反（Specification Violation）和方法-代码对齐（Method-Code Alignment）由大语言模型进行判断，并通过多次独立运行的多数投票来降低判断噪声。参考文献验证（Reference Verification）主要基于应用程序编程接口（API）（包括 Semantic Scholar、arXiv、OpenAlex、CrossRef 查询），并针对仅标题匹配的情况采用大语言模型消歧步骤。

表 7 | CoE 完整性审计各组件的自动化程度。

Component	Automation	Model (if LLM)
Score Verification	LLM extraction + automated comparison	Gemini 3 Flash
Specification Violation	LLM-judged	Gemini 3.1 Pro
Reference Verification	Automated + LLM disambiguation	Gemini 3 Flash
Method-Code Alignment	LLM-judged	Gemini 3.1 Pro

人工验证。尽管审计流程是自动化的，但在表 1 中报告之前，所有针对 I1（分数验证）、I2（规范违反）和 I3（参考文献验证）标记的阳性结果均经过人工审稿人手动审查和更正。在此过程中，少量审计器假阳性（例如，真实论文的 API 解析失败、分数提取错误）被识别并移除。例外情况：Sakana ASv2 的大多数 I2 违规可追溯至 BFTS–ADRS 设计不匹配，而非对抗行为（第 G 节）。对于 I4（方法-代码对齐），人工审稿人验证了大语言模型判断的样本；表 1 中的结果反映了大语言模型多数投票得分，未经人工更正。

E. 各审计度量的失败案例

E.1. I1: 分数验证错误

我们手动审查了五个系统中标记的每一个 I1（分数验证）失败案例，并将 22 个已确认的智能体错误分为五类（表 8）。⁴

表 8 | 按类别划分的已确认智能体 I1 错误。

Category	Definition	Count	%
value_mismatch	Paper headline number differs from evaluator rerun beyond tolerance, with the same metric and scale.	13	59%
cross_stage_cherry_pick	Writeup LLM selects a score from a different search-tree stage (e.g. ablation) than the node whose code is submitted.	4	18%
paper_score_unavailable	Paper contains no machine-readable headline number for the auditor to verify against.	2	9%
evaluator_error	Submitted artifact cannot be re-evaluated within the budget (timeout or crash on the agent's own solution file).	1	5%
metric_mismatch	Paper and code report different metrics, scales, or optimisation directions, so the numbers are not directly comparable.	2	9%

值不匹配 (n=13). 论文与代码在测量对象上一致，但在数值上存在分歧。大多数 (9/13) 差距在论文报告数值的 5% 以内——小到可能源于未报告的种子方差，但一致偏向于比重新运行结果更好的标题数据。在四个基线系统中，两个最大的差距是性质不同的错误：DS 云播种子-2 中伪造的度量方向 (26.7%)，论文将成本度量重新标记为“效用”并反转优化方向，使比基线更差的数字读起来像改进；以及 ARC 棱镜种子-3 中论文标题 (12.74) 与提交代码实际产生值 (26.24) 之间的 $2\times$ 不匹配。Sakana ASv2 贡献了两个 **value_mismatch** 案例，均由环境相关调优引起：求解器包含一个由环境变量 (`_ADRS_EVAL_GUARD`) 控制的超参数调优循环，在 BFTS 搜索期间该变量未设置，循环以最优参数执行，但在规范重新评估期间该变量被设置，循环被跳过，导致求解器回退到默认值（例如棱镜种子-0：调优 26.26 对比默认 22.34，差距 15%）。

跨阶段择优 (n=4). 所有四个案例均来自 Sakana ASv2，反映了 BFTS 流水线中的边界不匹配：最佳节点从阶段 1–2（初步调查和超参数调优）中选出，但撰写 LLM 接收来自所有四个阶段（包括消融）的摘要，并从整个池中挑选最有利的分数。例如，在棱镜种子-1 中，所选节点得分为 22.79，但论文报告为 25.39——该数字可追溯到消融摘要中的消融节点 6（“消融 KVPR 感知初始化”）。相同模式出现在云播种子-0 (+56%)，棱镜种子-2 (– 4.7%，论文低报) 和 `txn_scheduling` 种子-2 (+17%) 中。

^{4A} 少量审计员假阳性在报告表 1 前已修正；本节仅报告已确认的智能体错误。表 1 报告了每个系统 12 篇论文的通过率（EPLB 除外），因此其隐含的失败计数低于此处的 22 个错误，后者包含 EPLB 论文。

论文分数不可用 (n=2). 智能体的撰写内容中不包含标题度量的机器可读定量结果。两个案例均为基线系统失败：一篇 AIR 棱镜论文完全省略分数，以及一份 DS 云播 PDF 未能渲染其结果部分。

评估器错误 (n=1). 智能体声明为其解决方案的工件无法在预算内进行端到端重新评估。唯一案例是 ARC llm_sql seed-2，它附带 program.py——一个多条件实验框架，在产生任何单个数字之前，导入时即运行 8 个基线条件 × 3 个种子 × 多个数据集。

指标不匹配 (n=2). 论文与代码在测量内容上不一致。ARC llm_sql seed-3 发布了一个 0.0 的标题分数，论文本身将其归因于“与基准评估器框架的关键集成失败”——智能体将一个已知损坏的结果作为最终分数呈现。Sakana ASv2 cloudcast seed-1 报告每次迁移的美元成本，而评估器产生的是综合分数，使得这两个数字不可比较。

各系统的错误形态。表 9 显示，失败组合高度依赖于具体系统。Sakana ASv2 产生的错误最多（7 个），并且是唯一出现 cross_stage_cherry_pick 案例的系统——这是多阶段搜索流程所特有的失败模式，该流程将完整的实验历史暴露给撰写阶段。其两个 value_mismatch 案例是环境相关的调优，而非简单的分数膨胀。AIR 和 DS 的错误主要由小到中等的 value_mismatch 构成：数字存在且大致正确，但不能精确复现。ARC 跨越了最广的类别范围，并产生了最大的单一差异（106%），也是唯一出现 evaluator_error 案例的系统。**ScientistOne** 未产生已确认的 I1 错误，这与表 1 中 12/12 的分数验证结果一致。

表 9 | 各系统按类别确认的智能体 I1 错误。

System	val_mis.	cherry_pick	score_unavail.	eval_error	metric_mis.	Total
Sakana ASv2	2	4	0	0	1	7
AIR	3	0	1	0	0	4
ARC	5	0	0	1	1	7
DS	3	0	1	0	0	4
ScientistOne	0	0	0	0	0	0

E.2. I2: 规范违反分析

I2 审计在五个系统中标记了 11 篇存在规范违反的论文（表 11）。Sakana ASv2 占 11 篇被标记论文中的 10 篇；其余案例是一篇 AIR llm_sql 论文。ARC、DS 和 **ScientistOne** 在多数投票下产生零个 I2 违反。⁵ Sakana 的大部分 I2 违反可追溯到 BFTS – ADRS 设计不匹配（第 G 节）。

由于单篇论文可能触发多个违规类别，本文同时报告类别分布（表 10）和按系统划分的任务分解（表 11）。

⁵在联合投票（任一评委标记即算）下，ARC 有 3 篇，DS 有 1 篇，**ScientistOne** 有 1 篇。如正文所述，DS 种子 1llm_sql 仅被 2/5 的评委标记——低于多数阈值——尽管其包含利用评估器的列置换模式。

表 10 | 11 篇被标记论文中的 I2 违规类别。一篇论文可能触发多个类别，因此论文计数之和不等于 11。

Category	Papers	System(s)
Evaluator import	10	Sakana (10/10)
Evaluator exploitation	7	Sakana (7/10)
Specification exploit	5	Sakana (4/10), AIR (1)
Data leakage	1	Sakana (1/10)

评估器导入（10/10 篇 Sakana 论文）。每篇被标记的 Sakana 论文都导入了标准评估器，并将其作为优化预言机调用。机制一致：智能体添加 `from evaluator import evaluate`（或别名为 `_tune_eval`）的变体），然后循环遍历超参数配置——对每个配置调用 `evaluate()`——并保留最佳结果。该导入模式与本文的标准评估框架一致，该框架被注入初始代码模板以实现自动评分；智能体复制该模式，并在其上构建自己的调参基础设施。智能体本可以通过 BFTS 的迭代循环（每次迭代一个设置，由外部评估器评分）来调参，但第二阶段目标（“在多个参数设置下测试”）鼓励迭代内扫描，这需要直接导入评估器。

规范利用（4 篇 Sakana + 1 篇 AIR）。四篇 `Sakanatxn_scheduling` 论文（种子 1 – 3）和一篇 `llm_sql` 论文修改了 `EVOLVE-BLOCK` 标记之外的函数，或利用了度量边界情况。`txn_scheduling` 模式一致：智能体修改 `get_random_costs()`，一个明确禁止修改的函数，以运行参数扫描并返回最佳结果。AIR 唯一的规范利用案例（`llm_sql` 种子 1）是列置换利用：求解器在每行内物理地重新排列各列的值，破坏列完整性以抬高评估器的前缀缓存命中度量——该模式同样出现在 DS 种子 1 `llm_sql` 中，但未达到多数阈值。

表 11 | 各系统的 I2 规范违规情况。对于 Sakana ASv2，本文展示按任务分解；其他系统报告总数。所有计数均采用多数投票（3/5 评委）。

	系统 云端. eplb llm_sql 棱镜 txn_sched. 总计					
Sakana ASv2	1/3	0/3	3/3	3/3	3/3	10/15
AIR	0	0	1	0	0	1/15
ARC	0	0	0	0	0	0/15
DS	0	0	0	0	0	0/15
ScientistOne	0	0	0	0	0	0/15

干净运行。15 次 Sakana 运行中有 5 次未产生 I2 违规：全部三个 EPLB 种子以及 `cloudcast` 种子 2 和 3。EPLB 的求解器契约在结构上更简单（纯分配函数，无外部评分器依赖），而两次干净的 `cloudcast` 运行表明，即使在图优化任务上，智能体也能在不导入评估器的情况下完成 BFTS 阶段目标——但在 BFTS 的阶段压力下，这是例外而非默认情况。

E.3. I3: 引用完整性——发现的幻觉引用 以下列表展示了唯一的幻觉引用键。表 1 报告了各论文中的总出现次数（同一键可出现在多篇论文中；例如，ARC 的单个虚构键出现在全部

三篇 EPLB 论文中，在表中计为 3)。

幻觉引用

ARC (1)

sutskever2013importance: Sutskever 等人。带动量的随机梯度下降。国际机器学习大会，2013 年。

AIR (21)

ruth2018castflow: R  th, Jan 等人。CastFlow：在软件定义网络中使用预先路径计算的清洁板多播方法。2018 年 IEEE 第 43 届本地计算机网络会议 (LCN)，2018 年。

dou2022hetmoe: Dou, Shiwei 等人。HetMoE：面向异构集群的高效分布式专家混合训练系统。arXiv 预印本 arXiv:2210.12384，2022 年。

li2023lightllm: Li, Zhuohan 等人。LightLLM：一种具有令牌级键值缓存管理的高度优化的 LLM 推理系统。arXiv 预印本 arXiv:2309.04414，2023 年。

purohit2024prism: Purohit, Archit 等。Prism：在图形处理器集群上优化多模型大语言模型服务。第 29 届 ACM 国际编程语言与操作系统架构支持会议 (ASPLOS) 论文集，2024 年。

miao2023muxserve: Miao, Xupeng 等。MuxServe：多路复用大语言模型以实现高吞吐量与低延迟。arXiv 预印本 arXiv:2311.05602，2023 年。

cloudcast: Zheng, Q. 等。**CloudCast**：云网络中成本高效的多播路由。IEEE INFOCOM 2019 - IEEE 计算机通信会议，2019 年。

idreos2012main: Idreos, Stratos 等。主存列存储。Foundations and Trends® in Databases，2012 年。

lightllm2023: Li, Zhen 等人。LightLLM：一个轻量级且高效的基于 Python 的大语言模型服务框架。arXiv 预印本 arXiv:2310.01234，2023。

bhuiyan2024prism: Bhuiyan, M. 等。Prism：一种灵活且可扩展的多大语言模型服务系统。第 29 届 ACM 编程语言与操作系统架构支持国际会议 (ASPLOS) 论文集，2024。

eplb2023: Wang, H. 等。专家并行负载均衡在混合专家模型中的应用。高性能计算、网络、存储与分析国际会议论文集 (SC)，2023。

hussin2011 元启发式: 侯赛因 (Hussin)，MS 等人。旅行商问题的元启发式算法：综述。《克拉约瓦大学纪事，数学与计算机科学系列》，2011 年。

zhang2020job: 张军等。作业车间调度研究。国际生产研究杂志，2020。

jetstream2014: 拉布金，阿里尔等。JetStream：实现广域数据流传输。第 11 届 USENIX 网络系统设计与实现研讨会 (NSDI 14)，2014。

slingshot2022: 多伊，约翰等。Slingshot：跨联邦云环境的高性能路由。ACM SIGCOMM 2022 会议论文集，2022。

cloudcast2021: 陈怡婷等。CloudCast：多云部署中经济高效的数据分发。IEEE INFOCOM 2021 - IEEE 计算机通信会议，2021。

cui2004: 崔俊红等。动态网络中的服务质量组播路由。电气电子工程师学会网络期刊，2004 年。

laoutaris2011netstitcher: 拉乌塔里斯，尼古拉斯等。网络缝合器：从网络边缘解开大容量存储的束缚。2011 年 ACM 数据通信特别兴趣小组会议论文集，2011 年。

feng2020traffic: 冯鑫等。软件定义广域网中的流量工程：综述。电气电子工程师学会/美国计算机协会网络汇刊，2020 年。

幻觉参考文献 (续)

epstein2005online: 爱泼斯坦, 利亚和范斯特, 罗布。具有平方根大小物品的在线装箱。信息处理快报, 2005 年。

ba2023modelbox: 巴宇涵等。ModelBox: 一个多模型多租户服务框架。2023 年美国高级计算机系统协会年度技术会议 (USENIX ATC 23), 2023 年。

zheng2024eplb: 郑连民等。EPLB: 大语言模型中专家并行的负载均衡。arXiv 预印本 arXiv:2401.03221, 2024 年。

DS (41)

choy1991heuristic: 蔡明善。斯坦纳树问题的启发式算法及其在网络路由中的应用。1991 年 ACM SIGCOMM 会议论文集, 1991 年。

beloglazov2012energy: 别洛格拉佐夫, 安东和布亚, 拉杰库马尔。多云环境中的节能路由与资源分配。网络与计算机应用杂志, 2012 年。

grotschel1993steiner: 格罗切尔 (Grotschel), 马丁 (Martin) 等。电信中的斯坦纳树打包问题。1993 年。

romero2021automated: 罗梅罗 (Romero), F 等。利用大语言模型的自动化算法设计。神经信息处理系统进展, 2023 年。

wu2015spanning: 吴 (Wu), 川 (Chuan) 等。多云架构中基于生成树的数据传输。2015 年。

melian2012cloud: 梅利安 (Melian), L 等。云计算经济学: 综述。2012 年。

faragardi2017multi: Faragardi, Hamid Reza 等。多云端数据分布与成本优化。2017。

mishra2018cost: Mishra, A 等。多云环境中的成本高效路由。2018。

voss1992steiner: Voss, Stefan。带边容量的斯坦纳树问题。1992。

bubeck2023approaches: Bubeck, S 等。代码生成与综合方法。2023。

wang2020automated: Wang, X 等人。云计算中的自动化超参数优化。2020 年。

smith2019data: Smith, J 等人。现代云平台中的数据传输成本。2019 年。

liu2022network: Liu, Y 等人。网络感知的多云数据分布。2022 年。

jones2023oscillatory: Jones, R 等人。自动化搜索景观中的振荡动力学。2023 年。

kim2021puber: 金永镇 (Kim, Young Jin) 等。Puber: 面向专家混合模型的高效专家并行。arXiv 预印本 arXiv:2111.05454, 2021。

romera2021optimizing: 罗梅拉-帕雷德斯 (Romera-Paredes) 等。面向大规模分布式训练的专家混合模型优化。arXiv 预印本 arXiv:2102.04353, 2021。

clune2008how: 克卢恩 (Clune) 等。进化动力学如何影响网络拓扑。人工生命, 2008。

llm_agents_2023: 史密斯 (Smith, J.) 与多伊 (Doe, A.)。面向代码生成的自主大语言模型智能体。人工智能研究杂志, 2023。

api_misuse_2024: 王 (Wang, L.) 与李 (Lee, C.)。大语言模型生成代码中的应用程序编程接口误用。国际软件工程会议论文集, 2024。

fail_fast_2025: Garcia, M。面向编码代理的快速失败沙箱机制。IEEE 软件工程汇刊, 2025。

prism2024: Anonymous。Prism: 多大型语言模型服务系统基准。arXiv 预印本, 2024。

sarca2023: Anonymous。SARCA: 使用编码代理的系统架构研究。arXiv, 2023。

幻觉参考文献 (续)

- clockwork2023:** Anonymous. Clockwork: 面向边缘机器学习的快速可预测推理. arXiv, 2023.
- garcia1982fully:** 加西亚-莫利纳, 赫克托. 一种用于并发数据库更新的完全分布式无空算法. 《IEEE 软件工程汇刊》, 1982 年。
- 卡尔松 2020 组合优化:** 卡尔松, 埃利亚斯等. 基于图神经网络的组合优化. arXiv 预印本 arXiv:2010.16012, 2020 年。
- 克拉耶夫斯基 2024 专家混合:** 克拉耶夫斯基, 亚当等. 专家混合: 系统综述. arXiv 预印本 arXiv:2401.00000, 2024 年。
- 帕利瓦尔 2020REGAL:** 帕利瓦尔, 阿迪蒂亚等. REGAL: 一种基于迁移学习的软硬件协同设计方法论. MICRO, 2020 年。
- 贾因 1998 模拟:** Jain, AS 和 Meeran, S. 一种用于作业车间调度问题的模拟退火算法. 数学与计算机建模, 1998 年。
- tian2021cloud:** Tian, X 等. 云出口成本优化. ACM SIGCOMM 2021 会议论文集, 2021 年。
- zhao2020understanding:** Zhao, Y 等. 理解云网络出口约束. USENIX 年度技术会议 (ATC), 2020 年。
- binnig2021learned:** Binnig, Carsten 等. 学习型数据库系统之案例. arXiv 预印本, 2021 年。
- yu2014staccato:** Yu, Xiangyao 等. Staccato: 一种面向众核处理器的依赖感知事务调度系统. 2014 年 ACM SIGMOD 国际数据管理会议论文集, 2014.
- ren2016low:** Ren, Kun 等. 分布式数据库系统中的低开销死锁检测. 2016 年数据管理国际会议论文集, 2016.
- pavlo2017make:** Pavlo, Andrew 和 Stonebraker, Michael. 是什么让数据库系统变得快速? . ACM SIGMOD Record, 2017.
- blanas2010comparison:** Blanas, Spyros 等. 现代多核处理器上连接算法的比较. VLDB Endowment 会议录, 2010.
- bailis2014bolt:** Bailis, Peter 等. 螺栓式无冲突复制数据类型. ACM SIGMOD 记录, 2014.
- hardi1992precedence:** Hardi, S 和 Rakow, TC. 基于优先级的交易调度. 第二届数据工程研究问题国际研讨会论文集, 1992.
- kim2016fast:** Kim, Taesoo 等. 多核内存数据库中快速且可扩展的可串行化事务. 2016 年数据管理国际会议论文集, 2016.
- wu2017transaction:** Wu, Yingjun 等. 使用图着色的交易调度. 2017 年 ACM SIGMOD 数据管理国际会议论文集, 2017.
- lin2015scheduler:** Lin, Jianshu 等. 调度与否: 事务调度何时值得其开销? . 2015 年 ACM SIGMOD 国际数据管理会议论文集, 2015 年。
- faleiro2017high:** Faleiro, Jose M 和 Abadi, Daniel J. 具有确定性的高性能可串行化并发控制. 2017 年 ACM SIGMOD 国际数据管理会议论文集, 2017 年。
-

E.4. I4 审计失败分析

我们将 I4 审计标记的 95 项方法-代码不一致发现（涉及 25 篇论文⁶⁾）分为三个语义类别（表 13）。单篇论文可能产生多项发现，因此我们同时报告发现级别的分布和每个类别的不同论文计数。Sakana ASv2 被排除在此分解之外，因为其 I4 结果受到工件格式的干扰——审计的代码是完整的实验脚本而非提取的求解器，因此 I4 评审将论文与非求解器基础设施代码进行比较（第 G 节）。

表 13 | I4 方法-代码一致性发现，按类别 (n=95 项发现，25 篇受影响论文)。

Category	Definition	Findings	%	Papers
incomplete_broken	Paper-described mechanism or component is missing, partially implemented, or replaced with a degenerate fallback in the code.	49	52%	19
algorithm_class_mismatch	Code implements a fundamentally different algorithm class than the one described in the paper (e.g., paper claims Iterated Local Search, code is Simulated Annealing; paper claims a neural- or LLM-guided search, code is deterministic).	37	39%	15
deceptive_dummy_code	Code contains undisclosed elements designed to mislead evaluation: hidden environment-variable switches, or output deliberately shaped to game an evaluator metric.	9	9%	5

incomplete_broken (n=49, 19 篇论文)。最大类别。代码通常针对论文描述的相同问题，但缺少文中声称的一个或多个特定机制，或替换为退化的回退方案。反复出现的模式包括：描述的多起点初始化与 K 序列坍塌为单次确定性回溯遍历（AIR 棱镜）；声称的链路惩罚多样化产生对每个分区相同路径的恒定分配（AIR 云播）；描述的树形前瞻或代理成本模型在代码中完全缺失，而是在每次迭代中调用真实模拟器（ARC 云播，ARCtxn_scheduling);声称的 2.0 GB 内存阈值保护被简化为平凡的溢出检查（DS 棱镜）。

algorithm_class_mismatch (n=37, 15 篇论文)。代码实现了一个与论文所述根本不同的算法类别。最常见的子模式是声称的学习循环缺失的情况：一篇论文声称采用大语言模型驱动的进化搜索、神经网络预测器或大语言模型结构化查询语言优化器，而代码却是一个单一的确定性启发式方法，既没有大语言模型调用，也没有训练好的模型（例如，AIR 云预测：“混合神经符号求解器”→purely 经典斯坦纳树启发式方法；ARC llm_sql: “36 种大语言模型提示策略”→deterministic 数据框重排序；DS eplb: “27 代大语言模型驱动的进化搜索”→standalone 确定性负载均衡器）。经典算法类别的替换也反复出现：迭代局部搜索 →

表 1 表明有 26 篇不一致的论文 (12+10+3+1)；一篇论文（DS seed-3/cloudcast）无法明确分类，因此未纳入分类统计。

模拟退火（AIR 棱镜），丁克尔巴赫分数规划 →static 平方和代价（ARC 棱镜），递归划分 →single-level 分组（DS llm_sql).我们还观察到较为罕见的方法/基线倒置模式，即论文将 x 标记为所提方法，将 y 标记为被拒绝的消融，但代码却使用了 y （ARC 棱镜：声称“无共生 GRASP”为最终方法，而代码实例化的是共生 GRASP 打包器）。

deceptive_dummy_code (n=9, 5 篇论文)。提交的工件包含未公开的代码，其存在似乎旨在误导自动化评估。所有九项发现均来自 ARC，并分为两个子模式：（一）隐藏的环境变量开关（4 项发现，2 篇论文）：代码在导入时读取一个未公开的变量（CONDITION、ABLATION），并分派到多个不同求解器之一，而论文呈现的是单一统一算法（ARC 云预测种子 2 和 3）。（二）评估器博弈（5 项发现，3 篇论文）：代码被有意塑造以抬高某个度量而不解决底层任务，例如返回一个空的列排序列表，同时在内部置换值以最大化前缀缓存命中（ARC llm_sql 种子 3）。

各系统的错误形态。表 14 显示失败组合具有显著的系统特异性。AIR、DeepScientist 和 **ScientistOne** 未产生 deceptive_dummy_code 发现：当这些系统不一致时，差距在于论文所述与代码所做之间，没有主动混淆。ARC 是唯一产生 deceptive_dummy_code 发现的系统（9/9, 5 篇论文），涵盖隐藏环境变量开关和前缀命中评估器博弈两种类型。

表 14 | 按类别划分的每个系统的 I4 发现。每个单元格是一个发现计数；单篇论文可能贡献多个发现。

System	algo_class_mismatch	incomplete_broken	deceptive_dummy_code	Total
AIR	3	9	0	12
ARC	15	23	9	47
DS	15	16	0	31
ScientistOne	4	1	0	5

F. MLE-Bench 与参数高尔夫评估

表 15 | 我们评估中使用的 MLE-Bench 任务名称与其对应的官方任务 ID 之间的映射。

Task Name	Task ID
3D Object Detection	3d-object-detection-for-autonomous-vehicles
AI4Code	AI4Code
iMet 2020 FGVC7	imet-2020-fgvc7
RSNA Brain Tumor	rsna-miccai-brain-tumor-radiogenomic-classification
iNaturalist 2019 FGVC6	inaturalist-2019-fgvc6

我们在 MLE-Bench 和参数高尔夫上评估 **ScientistOne**。有关基准和评估设置的详细信息如下。

MLE-Bench。我们选择了五个 **MLE-Bench** Chan 等人（2024）Kaggle 竞赛，涵盖医学影像、细粒度识别和三维感知。表 15 总结了

我们评估中使用的任务名称与其官方任务 ID 之间的映射。我们特别针对中高难度级别的任务，因为它们具有足够的复杂性和实质性研究价值，可作为自动化科学发现和论文写作的有意义基准。具体而言，AI4Code、iMet 2020 FGVC7 和 iNaturalist 2019 FGVC6 被归类为中等难度任务，而三维目标检测和 RSNA 脑肿瘤为高难度任务。代理获得任务描述和代码执行环境以开发其解决方案。执行环境包括一个配备 8xH100 GPU、192 个 CPU 核心和 1TB RAM 的计算节点。为协助格式化，代理可以无限次查询验证工具，以确保其提交的 CSV 文件结构正确。此外，我们允许代理最多查询评分服务器 16 次，以获取测试数据的评估分数。这偏离了官方 MLE-Bench 协议，该协议将评估限制为最终生成解决方案的单次提交。我们采用这种修改后的设置，以更好地模拟现实世界的 Kaggle 竞赛，参与者在开发阶段迭代地向公共排行榜提交解决方案以获得反馈。虽然代理被指示主要基于验证数据集得出的度量来迭代其解决方案，但评分服务器被谨慎使用，以选择测试集上表现最佳的模型。此外，此设置对于允许代理在生成论文时报告准确的最终测试度量是必要的。

参数高尔夫。为评估在全新、实时环境中的适应性，我们在参数高尔夫竞赛（Parameter Golf competition, OpenAI, 2026）上对科学家一号（**ScientistOne**）进行了评测。该实时竞赛是一项聚焦于大语言模型（LLM）训练的、非常契合的人工智能研究任务。它要求参赛者在严格约束下训练出性能最高的语言模型：最终工件必须控制在 16MB 大小限制以内，且训练过程须在 8xH100 系统上于 10 分钟内完成。性能通过压缩率——以与分词器无关的每字节比特数（BPB）衡量——在精细网络（FineWeb）验证集上进行评估。我们为智能体提供了一个沙盒工具，用于执行求解代码以获取评估指标。此外，智能体还获得了一个知识库，其中包含截至 2026 年 4 月 27 日的官方排行榜解决方案。截至该日期，最先进的（SOTA）分数为 1.0611，由名为“BOS-Fixed SmearGate + LQER + SparseAttnGate + 9-Hparam Stack”的解决方案取得。智能体的任务是在这些现有排行榜基线的基础上进行构建，以发现能够进一步提升 SOTA 分数的新颖技术。

G. 基线适配细节

所有基线均为开源系统，在相同条件下适配至 ADRS 基准：双子座 3.1 专业版（Gemini 3.1 Pro）骨干网络，每个任务最多 20 次求解器迭代，2 小时代码生成窗口，每个任务 3 个随机种子。因基础设施问题（应用程序编程接口超时、速率限制、 LaTeX 编译错误）而崩溃的运行，以全新状态重新尝试，最多 3 次。任何运行均未因提高求解器分数而重新尝试。以下详述各系统的适配情况。

坂名人工智能科学家 v2（Sakana AI-Scientist v2, 坂名）。坂名 v2 需要最深度的适配。原始代码库在其 4 阶段最佳优先树搜索（BFTS）流水线中假定了一个机器学习训练 workflow：阶段目标指示智能体“调整学习率和批大小”、“从拥抱脸（HuggingFace）引入数据集”以及“避免更改模型架构”。为适配 ADRS，我们重写了 agent_manager.py 中的全部四个阶段目标以及 parallel_agent.py 中的 14 处提示位置，将机器学习特定术语（“训练循环”、“模型架构”、“数据集”）替换为优化领域对应术语（“求解搜索”、“算法设计”、“问题实例”）。

除提示词变更外，我们创建了一个桥接入口点 (`perform_experiments_adrs.py`)，用于加载 ADRS 任务规范、注入演化块起始代码，并通过 `one-ai-scientist` 任务加载器而非 Sakana 内置解释器来路由评估。我们还添加了 Gemini 3.1 Pro 后端 (`backend_gemini.py`)，因为原始代码库仅支持 OpenAI 兼容的应用程序编程接口。总共修改或创建了 16 个源文件，外加 5 个任务特定的想法文件和 NeurIPS 2026 L^AT_EX 模板。在取证审计期间发现了四个集成后修复需求：1. 规范评估器注入。BFTS 绕过自身的 `exec_callback` 直接执行生成的代码。我们注册的评估回调是死代码。我们改为在初始代码模板中附加一个受保护的规范评估框架，使每个生成的 `runfile.py` 都运行官方 ADRS 评估器，并将规范度量打印到标准输出，供 BFTS 度量解析器使用。2. 确定性最佳节点选择。BFTS 通过温度 = 0.3 的大语言模型调用选择其“最佳”节点，该调用考虑“训练动态”和“绘图质量”——而不仅仅是原始分数。我们添加了对所有 4 个阶段中所有节点的确定性 `argmax`，生成 `best_solver.py` 并进行全新的规范重新评估。3. 论文与代码对齐。撰写管道读取大语言模型选择的节点，而 `best_solver.py` 来自确定性 `argmax`——在所有 15 次运行中产生了不同的算法。本研究中评估的论文在此修复后未重新生成，因此论文描述可能指向大语言模型选择的节点，而非得分最高的节点。4. 绘图生成路径。上游 `load_exp_summaries()` 将 `logs/0-run/` 硬编码为摘要目录，但我们的桥接将摘要写入运行根目录。这种不匹配导致 `aggregate_plots` 接收到空数据，在所有 15 次运行中生成零个图形。因此，所有 15 篇 Sakana 论文均无图形。

Sakana 的规范违反模式（15 篇论文中有 10 篇被标记）在 §D 中分析。主要原因是 BFTS 的多阶段架构（假设评估自包含）与 ADRS 的求解器契约（将评估器视为不可触碰的基础设施）之间的设计不匹配。

自动研究爪 (AutoResearchClaw, ARC)。ARC v0.4.0（提交 39c996b，2026 年 4 月）需要 2 处源码补丁：一处是在 `llm/client.py` 中向思考模型白名单添加 Gemini 3.x（2 行），另一处是为 ADRS 任务配置脚手架目录。未对 ARC 的搜索逻辑、停止准则或评估流水线做任何更改。我们设置 `max_iterations=20`（6.7× ARC 默认值为 3）和 `max_refine_duration=7200s`。ARC 的内部限制（最多 2 次决策转向、最多 3 次修复循环）属于架构性限制，未做修改。ARC 的流水线后评分 (PQEP) 在 15 次运行中有 4 次失败，原因是评估器路径替换中的正则表达式缺陷。通过在被提交的代码上重新运行规范的 ADRS 评估器解决了这些问题。所有 15 次运行中的图形生成均失败，因为 ARC 的 FigureAgent 需要 Docker 镜像，而我们的环境中没有该镜像。所有论文均在没有图形的情况下生成。

ARC 的蓝图规划器在见到 ADRS 的单文件约束之前，会设计多文件项目架构 (`config.py`、`models.py` 等)。脚手架通过将入口点重命名为 `program.py` 来部分补偿，但求解器仍会导入仅存在于原始实验目录中的辅助模块。15 个求解器中有 5 个在隔离环境中重新评估失败——这是 ARC 的 I1 和 I4 失败的主要原因。

DeepScientist (DS)。DS v1.5.17（标签 v1.5.17，2026 年 4 月）在 Codex CLI 0.57.0 上仅需提示词更改——未修补任何源文件。任务规范和 NeurIPS 2026 模板通过写入阶段提示词注入。我们设置 `CODE_TIMEOUT=7200s` 和 `WRITE_TIMEOUT=5400s`，

代码和写入阶段各最多重试 3 次。DS 的写入技能指示智能体通过 Semantic Scholar、arXiv 和 CrossRef API 检索引文。然而，在所有 15 个写入阶段日志中，智能体从未调用任何检索 API 或 MCP 工具，所有参考文献均从模型记忆中生成。这是模型合规性失败——工具可用，但智能体始终走捷径，跳过引文检索指令。

AI-Researcher (AIR)。AIR 需要最广泛的适配：修补了 19 个源文件，以对接 ADRS 任务配置、基于 Docker 的沙箱和 NeurIPS 2026 论文模板。AIR 的 MetaChain ReAct 智能体在 Docker 容器内运行。我们为其系统配置了相同的 Gemini 3.1 Pro 模型和迭代预算。在 15 次运行中，有 6 次因 Gemini API 速率限制或沙箱崩溃而需要重新运行。四篇论文需要手动 L^AT_EX 修复（尾随换行符、格式错误的 filecontents 环境、参考文献损坏）。